

TOPST 단기 교육 과정

## TOPST 임베디드·AI 통합 제어 실습 과정 (3일)



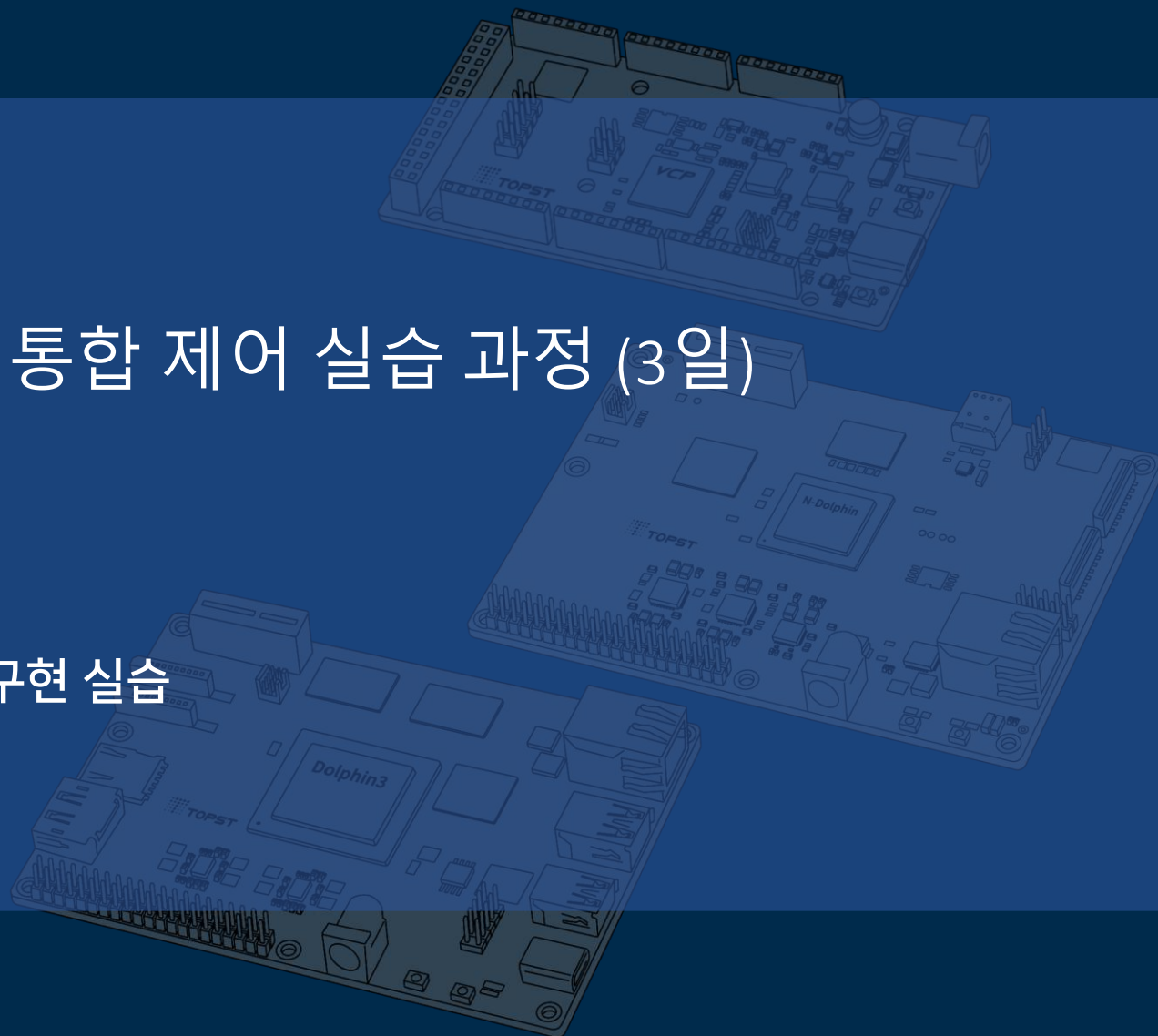
3일차

CAN 기반 제어 시스템 구현 실습

Telechips



TOPST



# 목차

## Table of Contents

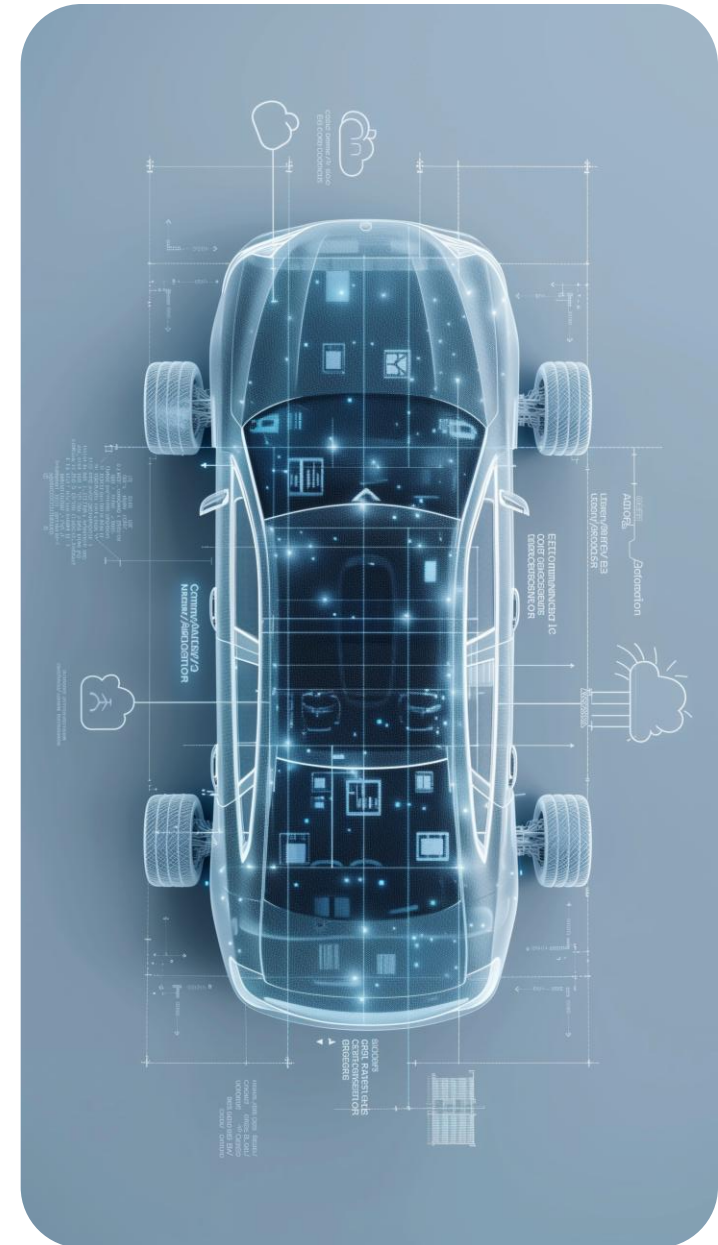
Introduction to CAN 01

D3-G CAN TASK

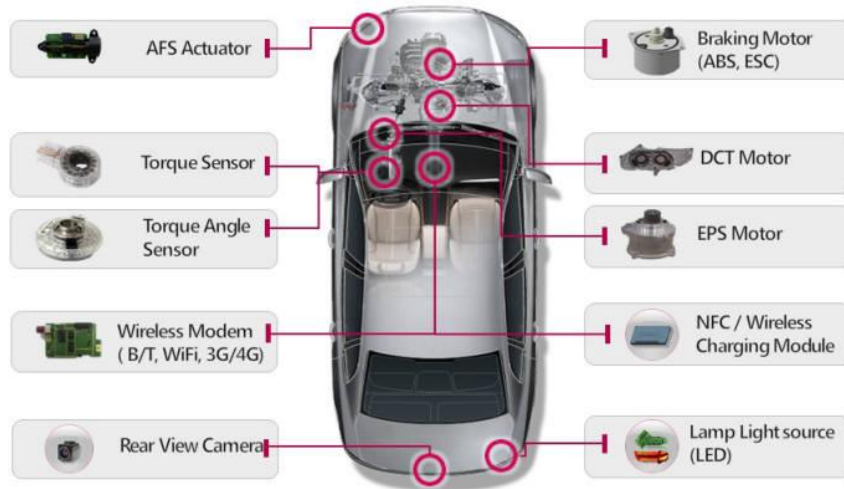
VCP-G CAN TASK 02

AI 추론 결과 연동 및 구조 이해 03

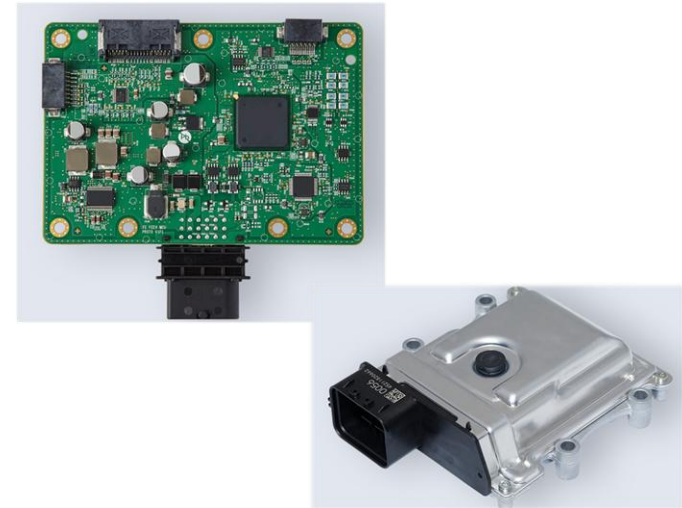
사용자 정의 제어 시나리오 구현 04



# 차량 내 전장장치들과 제어기



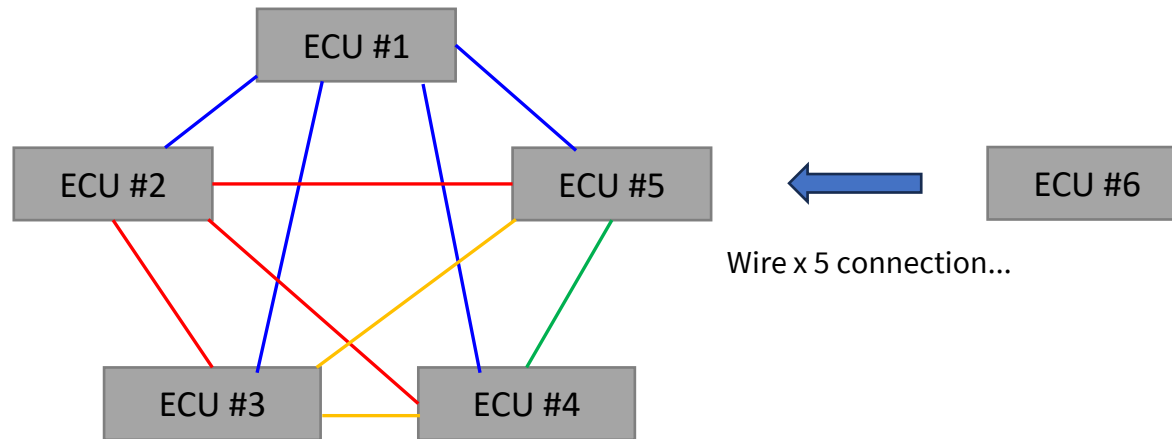
전장장치들의 예시



제어기의 예시

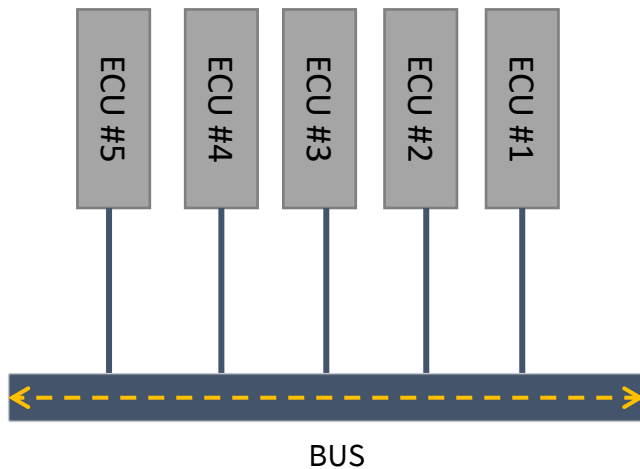
- 전자장치 : 전기&전자적으로 동작하는 차량 부품(ex. 액추에이터, 모터, 센서 등..)
- 제어기(Controller) : 차량 내에 구성된 수많은 장치들을 제어할 수 있는 물리적인 하드웨어
- 기능이 고도화될 수록 전자장치의 수가 증가되며, 이를 제어하는 제어기의 역할이 중요함
- 각 장치들끼리 통신을 해야 하는 기술적 수요가 필연적임
- 제어기끼리 통신을 한다면 과연 어떻게 해야 할까.

# ECU Wire Connection



- 만약 ECU의 통신라인이 위와 같은 구조라면 ??
- 제어기가 추가될 때마다 통신라인 추가
- 비용과 함께 차량 무게 증가
  - 생산성 & 안전성 부적합

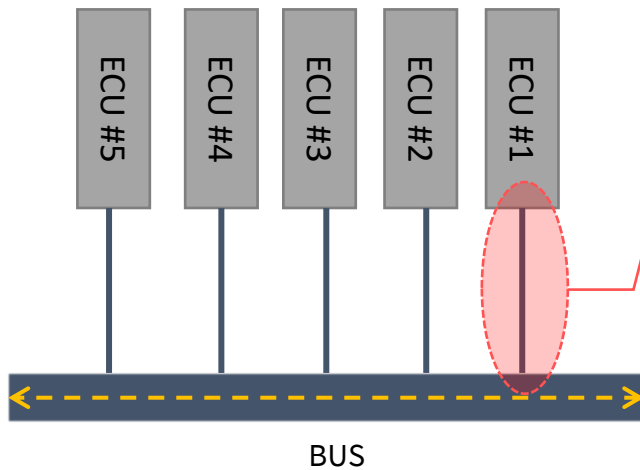
# ECU Wire Connection



- CAN (Controller Area Network)
  - 제어기 간 네트워크 효율화를 위해 설계 및 개발(BOSCH)
  - Broadcasting 방식의 network (src/dst 지정x)
  - 현재까지 모든 OEM에서 차량 네트워크로 CAN 통신을 채택함

|      |                                                                    |
|------|--------------------------------------------------------------------|
| 1983 | Bosch社에서 차량용 네트워크 프로토콜 개발 프로젝트 착수                                  |
| 1986 | CAN 프로토콜이 Society of Automotive Engineers (SAE)에 공식적으로 소개됨         |
| 1987 | Intel社와 Philips社에 의해 첫 번째 CAN 칩이 생산됨                               |
| 1991 | Bosch, CAN 2.0 사양 발표                                               |
| 1992 | Mercedes-Benz社에서 최초로 CAN 적용 차량 생산                                  |
| 1993 | ISO 11898 (data link and high-speed physical layer) 사양 제정          |
| 1994 | SAE J1939 사양 제정                                                    |
| 1995 | ISO 11898 사양 개정 (extended frame format)                            |
| 2003 | Data link 계층과 high-speed physical 계층 사양 분리 (ISO 11898-1 / -2)      |
| 2006 | ISO 11898-3 (low-power, low-speed physical layer) 사양 제정            |
| 2007 | ISO 11898-5 (low-power, high-speed physical layer) 사양 제정           |
| 2011 | CAN FD 프로토콜 개발 착수                                                  |
| 2013 | ISO 11898-6 (physical layer with selective wake-up function) 사양 제정 |
| 2015 | ISO 11898-1 (Classical CAN and CAN FD) 사양 발표                       |

# CAN Connector



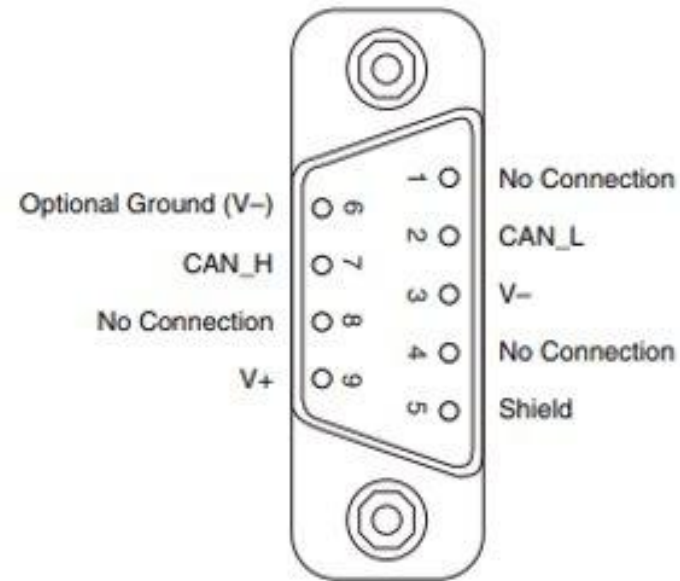
- 제어기끼리 BUS와 통신을 할 땐, 어떤 커넥터를 사용?
  - USB C type?
  - USB A type?
  - ...
  - 만약, 서로 다른 제어기 회사끼리 자신들이 만든 커넥터를 사용한다면, 커넥터의 호환성 문제가 발생



# CAN Connector



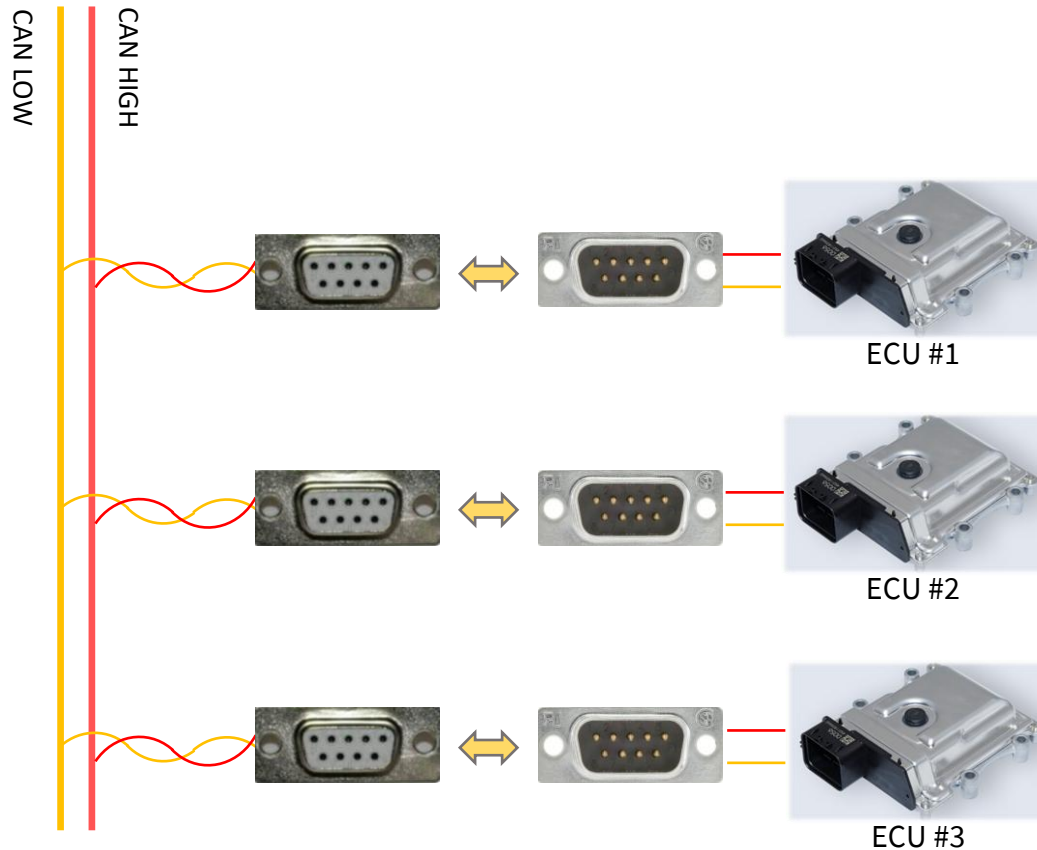
D-Sub 9pin Connector



D-Sub 9pin Connector 구조

- CAN 통신을 위한 포트로서 D-Sub 9 pin 커넥터 사용을 **약속**
- D-Sub 포트에 연결되는 connect hole도 약속
  - 2번 - CAN Low signal을 위한 wire
  - 7번 - CAN High signal을 위한 wire
- 대부분의 제어기 & 디버깅 장비는 D-sub 9 pin 포트 지원을 함

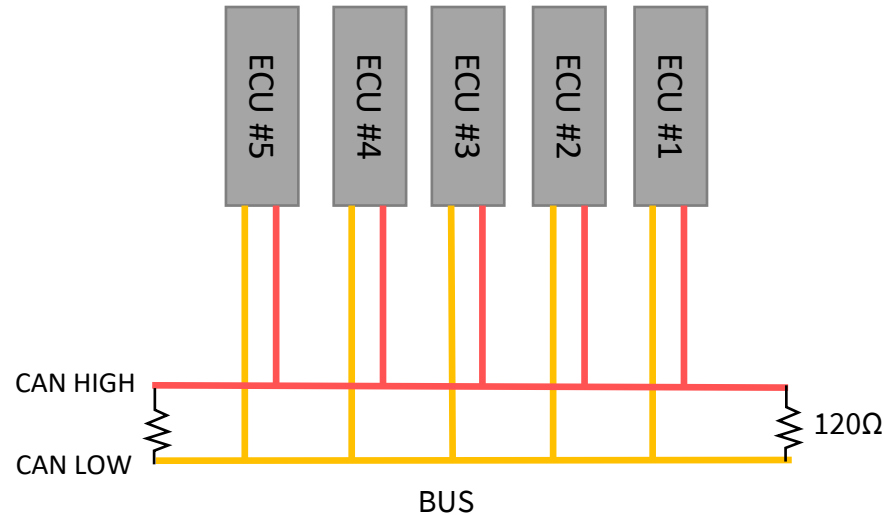
# CAN Connector



CAN connection

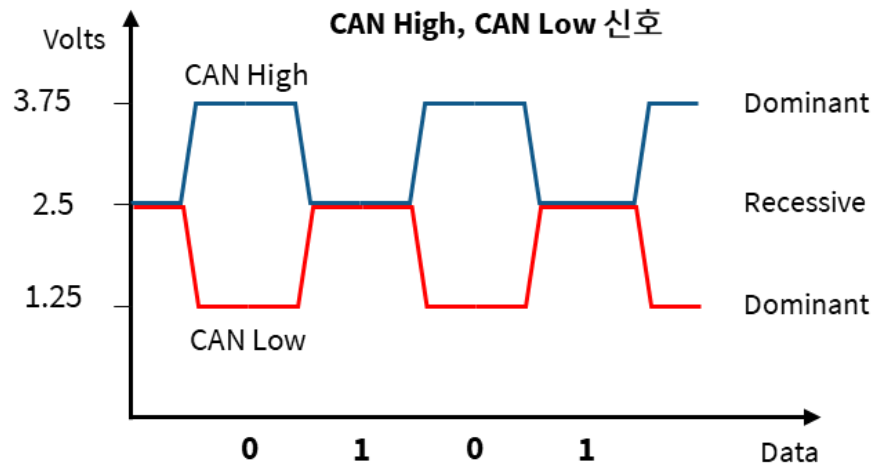
- 여러 개의 제어기끼리 통신한다면,
- CAN High, CAN Low 시그널의 버스 안에서,
- 통신에 참여하고 싶은 커넥터에 각각 연결하게 되면 CAN 통신 가능

# CAN High & Low



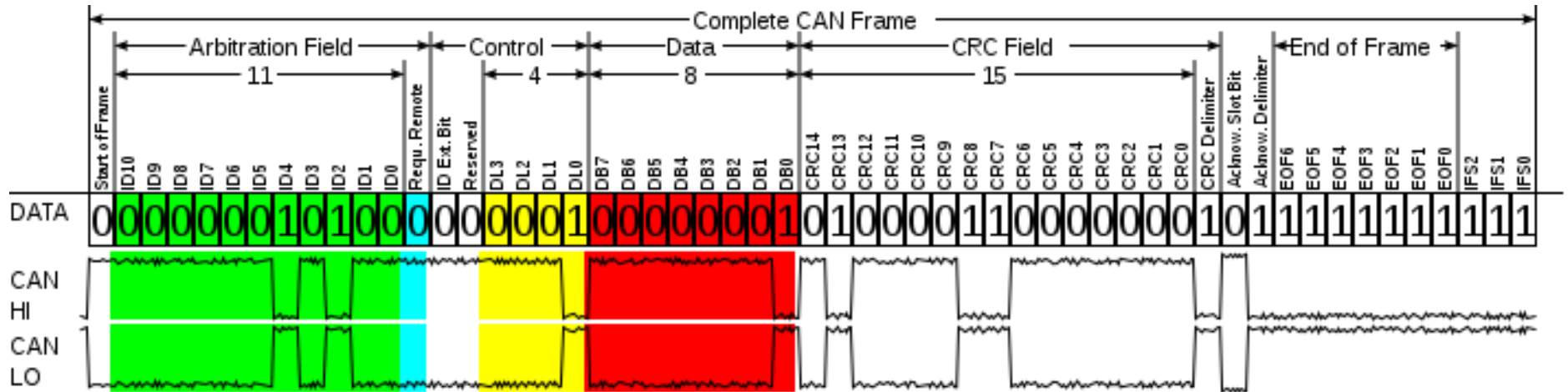
- 통신에서 메시지를 보낸다는 것은 전선에 어떤 전기적인 신호를 발생시켜 전달함을 의미
- Computer 체계에서 모든 정보는 binary 정보( 0과 1 )로 표현된다.
- CAN 통신은 특별히 HIGH와 LOW 두 개의 라인으로 0과 1을 표현함
- 그렇다면 과연 어떻게 0과 1로 표현할 수 있을까

# CAN High & Low



- [RULE] CAN High에 걸리는 전압과 Low에 걸리는 전압의 차를 이용해서 0,1 을 표현한다.
  - CAN High - CAN Low = 0.9 ~ 5 V → 0 (Dominant)
  - CAN High - CAN Low = -0.1 ~ 0.5 V → 1 (Recessive)
- CAN High 와 Low 파형은 서로 완벽하게 대칭하는 특성을 보임
- Dominant (Data 0) 가 Recessive (Data 1)보다 우선 순위가 높음 ( 중요 )
  - 여러 대수의 제어기가 하나의 버스로 연결이 되어 있다 가정
  - 모든 제어기가 동시에 신호를 보내는데, 우선순위를 어떻게 처리하는지에 대한 가이드가 정의되어야 함
  - 결과적으로 버스에는 Dominant가 우선순위가 높기 때문에 버스 전체에는 Dominant에 해당되는 전압이 출력 된다.

# Overview of CAN Message Format



- CAN 에서 모든 메시지 전송은 브로드캐스트 방식으로 이루어짐
- 송신되는 메시지 하나는 위와 같이 구성되어 있음
- CAN 에서는 Source Address, Destination Address가 없음
- 따라서, CAN message 에서는 ID 영역이 존재
  - Arbitration Field 중 ID에 해당하는 비트가 총 11개 존재

# CAN DB

| Message ID | Message Name    | Data Length (byte) | Transmitter         | Receiver            | Period |
|------------|-----------------|--------------------|---------------------|---------------------|--------|
| 0x123      | Steering_status | 4                  | Steering Controller | Module A controller | 100ms  |
| 0x145      | Engine_status   | 3                  | Engine Controller   | Module B Controller | 200ms  |
| 0x167      | Battery_status  | 4                  | BMS                 | Module C Controller | 100ms  |

CAN DB Example

- CAN 메시지 송신자, 수신자를 구분하기 위해 메시지 ID를 사용함
- OEM에서 ID별로 메시지 이름을 정하고, 해당 메시지의 송수신 제어기가 어디인지 정의하며 이런 유형의 모든 내용이 정리된 데이터베이스를 CAN DB라고도 부른다.
- 보통 ID는 16진수로 표현됨
- CAN 메시지는 네트워크 안에서 각각 고유한 ID를 보유하는 것이 원칙이다.
  - 다만, 차량 내부에는 여러 CAN 네트워크 영역이 존재하며,
  - CAN DB도 서로 별도로 존재하여 네트워크 영역 간 서로 관련 없는 독립적인 정보이다.

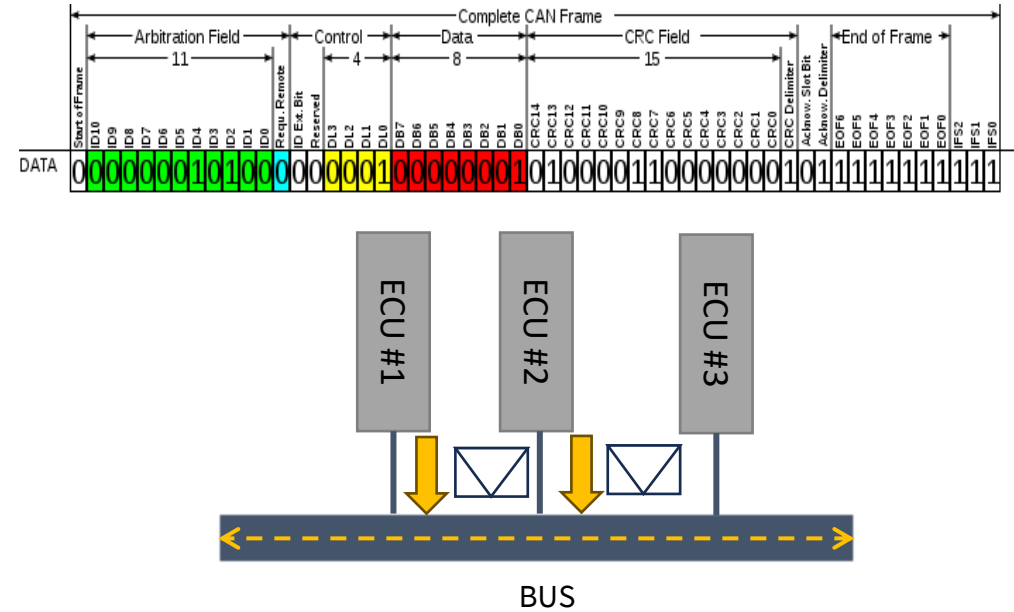
# Message's Priority

| Message ID | Message Name    | Transmitter         | Receiver |
|------------|-----------------|---------------------|----------|
| 0x123      | Steering_status | Steering Controller | xxx      |
| 0x145      | Engine_status   | Engine Controller   | xxx      |
| 0x167      | Battery_status  | BMS                 | xxx      |

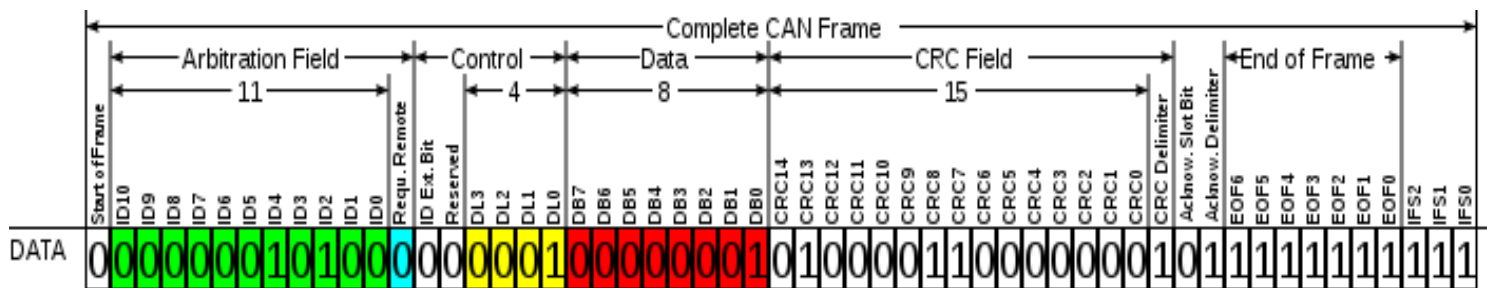
0x123 = 00100100011

0x145 = 00101000101

0x167 = 00101100111



- CAN 통신에서는 특정한 순간에 버스 전체에 오직 하나의 제어기가 하나의 메시지만 수신할 수 있음
- ECU1과 ECU2에서 동시에 메시지를 보내려고 한다면?
  - 한번에 여러개의 메시지를 보낼 수 없는 CAN 통신 특성상, 메시지의 우선순위를 따진다
  - Data 0 = Dominant , Data 1 = Recessive
  - Priority : Dominant > Recessive
  - ID값이 작을수록 우선순위가 높다.
  - 각각의 제어기들은 현재 네트워크에 다른 제어기가 메시지를 보내고 있는지 아닌지를 계속 체크

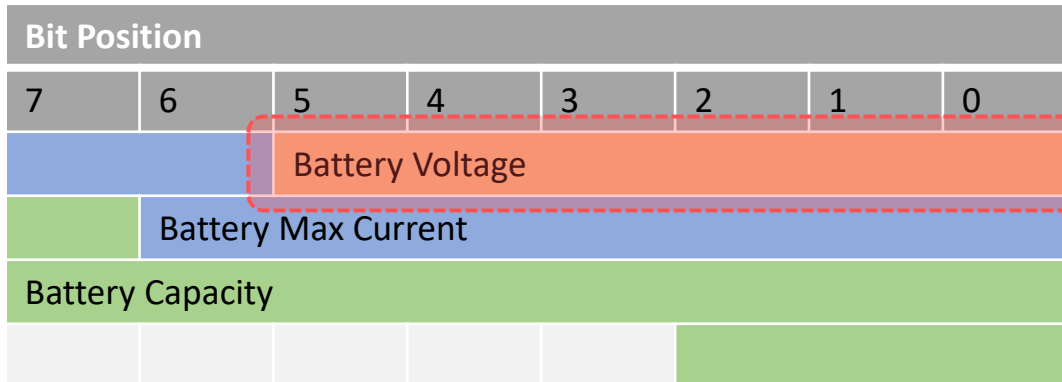


- <sup>14</sup> Confidential and Proprietary, Telechips Inc.

# Signal & Data length Code

| Message ID | Message Name    | Data Length (byte) | Transmitter         | Receiver | Period |
|------------|-----------------|--------------------|---------------------|----------|--------|
| 0x123      | Steering_status | 4                  | Steering Controller | xx,xx,xx | 100ms  |
| 0x200      | Engine_status   | 3                  | Engine Controller   | xx,xx,xx | 200ms  |
| 0x300      | Battery_status  | 4                  | BMS                 | xx,xx,xx | 100ms  |

CAN DB Example



Message Signal Example

Message Signal Example

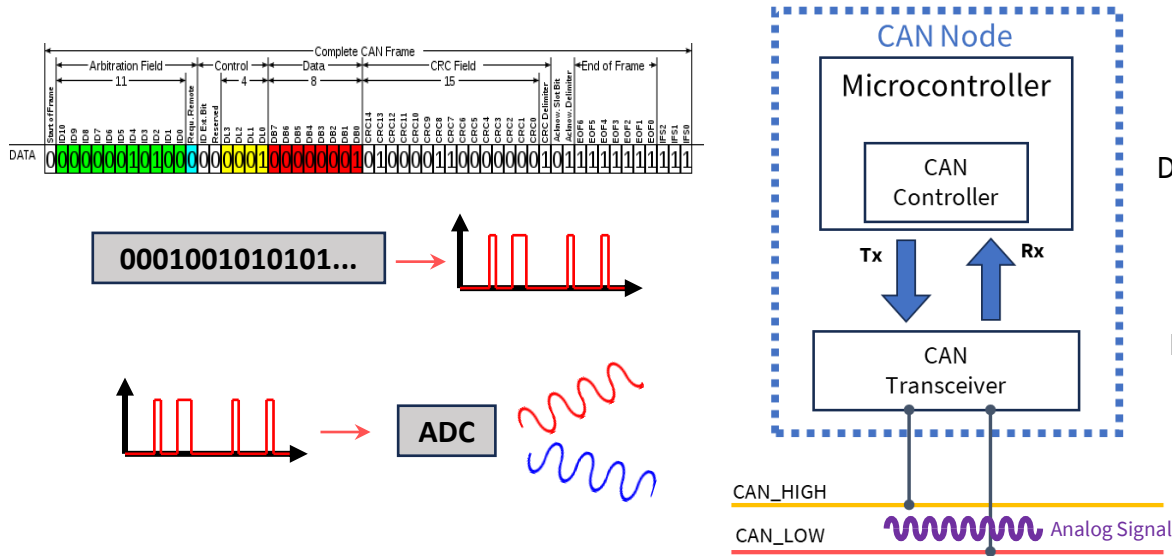
| Message ID | Signal List         |
|------------|---------------------|
| 0x300      | Battery Voltage     |
|            | Battery Max Current |
|            | Battery Capacity    |

Battery Voltage Signal Detail Information

- Start bit : 0
- Length : 6
- Unit : V
- Offset / Factor : xxx/xxx

- ID가 0x300 인 메시지는
  - Battery 정보를 담는 CAN 메시지
  - Battery Voltage, Max current, Capacity 시그널을 포함
  - DLC : 4Byte

# CAN Message 전송 흐름



Data Link Layer  
ISO 11898-1

Physical Layer  
ISO 11898-2/  
11898-3

**ISO 11898-1:** CAN 데이터 링크 계층을 정의한다. CAN 프로토콜에서 데이터의 전송 및 오류 처리 기능 뿐만 아니라 표준 CAN 메시지의 구조와 프레임의 형식을 규정한다.

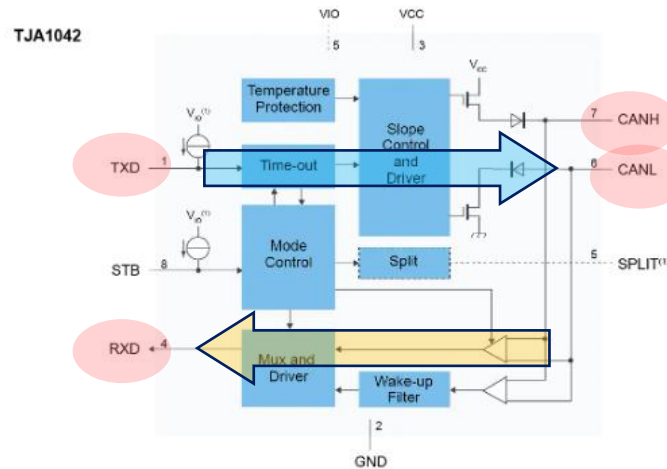
**ISO 11898-2:** 고속 CAN(CAN High-Speed)의 물리 계층을 정의한다. 차량 내 통신에서 주로 사용되며, 데이터 전송 속도는 최대 1 Mbps이다.  
**ISO 11898-3:** 저속 CAN(CAN Low-Speed)의 물리 계층을 정의한다. 주로 차량의 간단한 네트워크 및 저속 통신에 사용되며 Fault Tolerant(내고장성, 결함 허용) 특징을 가지고 있다.

- CAN 통신과 연관된 동작을 하는 장치 : CAN controller
  - CAN controller 는 MCU 내부에 마련되어 있음
  - CAN 통신의 프로토콜을 따라서 실제로 CAN 통신을 할 수 있도록 지원
- CAN 컨트롤러에서 링크 계층 표준을 만족하며, 데이터의 전송, 수신 및 프로토콜을 담당한다.
- CAN Transceiver( **T**ransmitter + **R**eciever)
  - 디지털 신호, 아날로그 신호를 서로 변환하고 송수신하는 역할을 지원한다.
  - Controller에서 생성한 신호(메시지)를 ADC를 통해서 전압값으로 변환
  - 변환된 전압 신호(Analog Signal)을 CAN bus로 전송

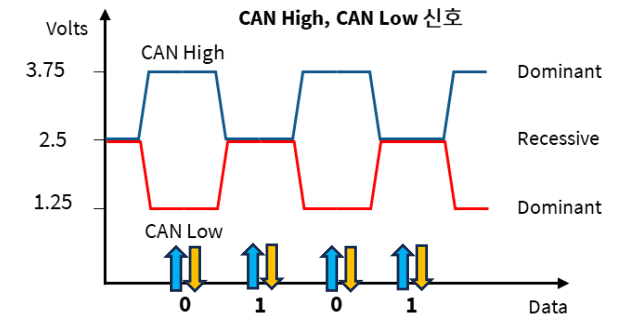
# CAN Transceiver



CAN Transceiver 칩 (NXP JTA1042)

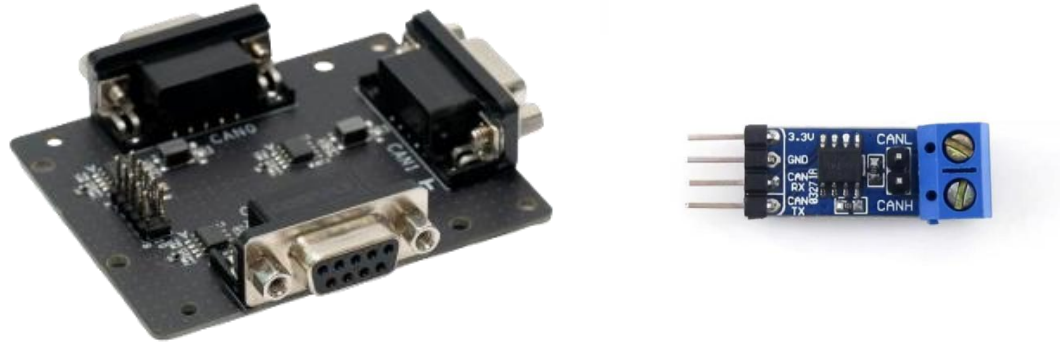


NXP JTA1042 회로도



- MCU는 3.75V, 2.5V, 1.25V 의 전압을 출력할 수도 없고, 반대로 전압을 읽어서 신호가 0인지 1인지 해석할 수 없음
- 따라서 CAN Transceiver( **Trans**mitter + **Recei**ver)는 CAN 통신에서 필수적으로 요구되는 장치
  - 디지털 신호,아날로그 신호를 서로 변환하고 송수신하는 역할을 지원한다.
    - 전위차를 이용해 0,1 신호로 변환
  - Controller에서 생성한 신호(메시지)를 ADC를 통해서 전압값으로 변환
  - 변환된 전압 신호(Analog Signal)을 CAN bus로 전송

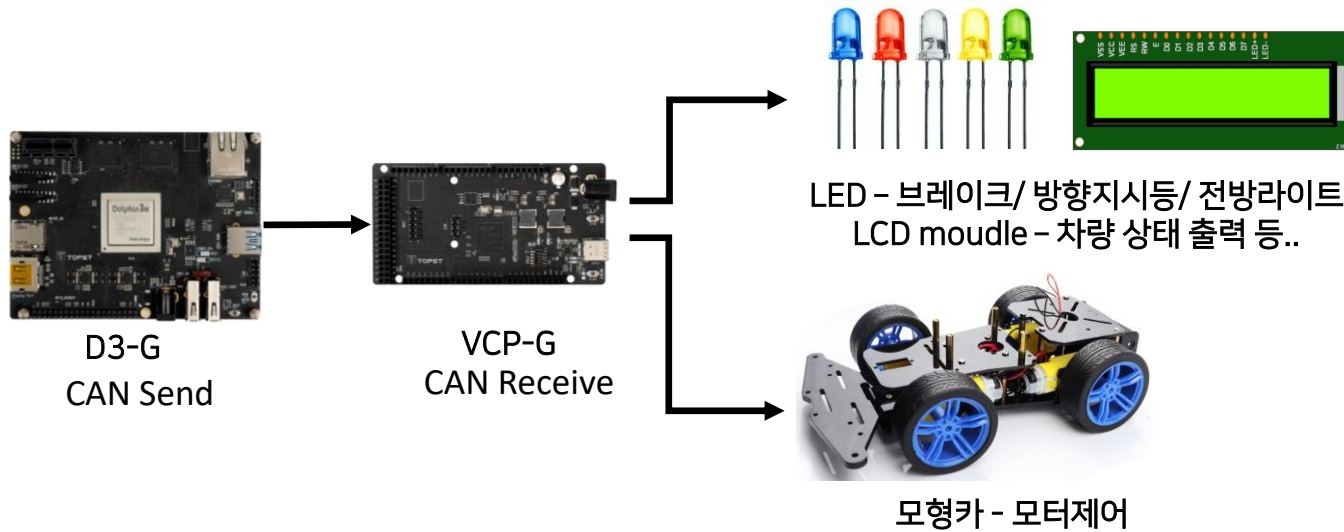
# CAN Transceiver



CAN Transceiver 장치들

- 대부분 Transceiver 는 D-sub 9pin 포트로 지원 혹은,
- 간단하게 High, Low 하나의 채널만 사용할 수 있게 pin header 로 지원

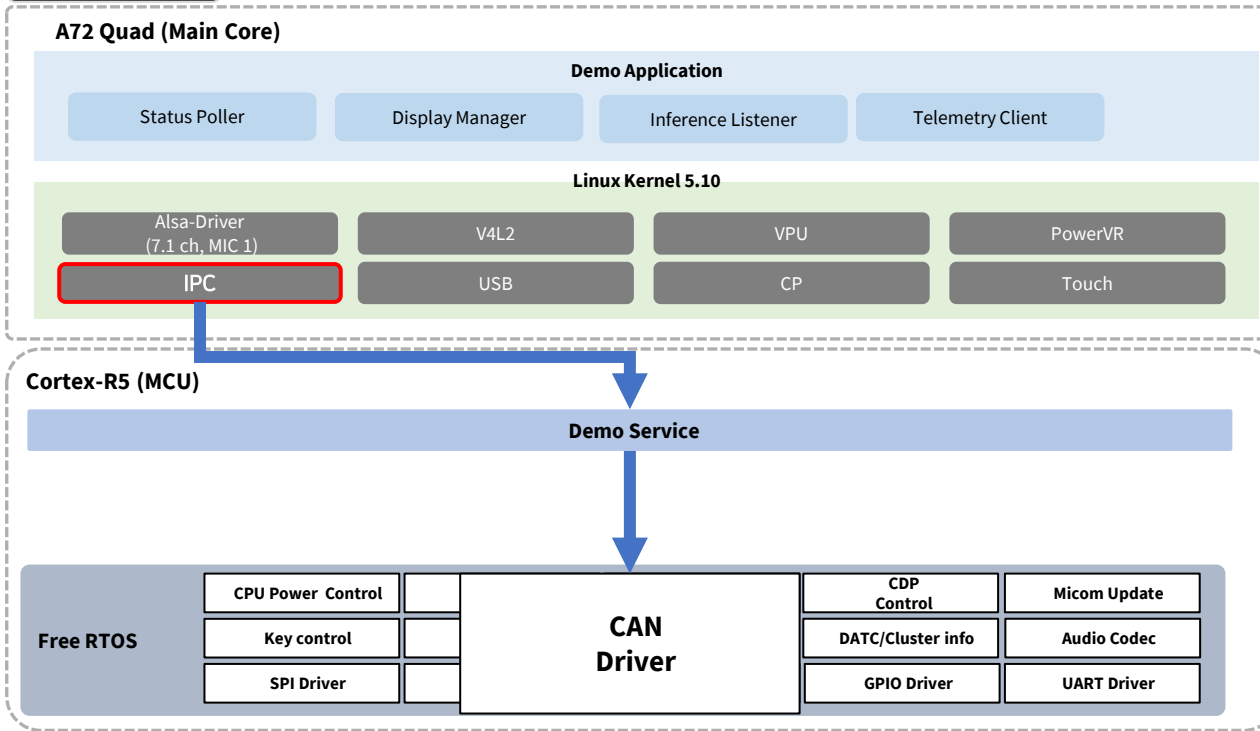
# CAN TASK 실습 구성도



- D3-G에서 CAN 메시지 송신
- VCP-G 에서 CAN 메시지 수신
- 수신된 결과에 따라 장치 제어

# D3-G CAN 통신 구조

## TOPST D3-G



### \*IPC : Inter Processor Communication

1. D3-G to MCU IPC를 이용해 상태 전달
  - CAN Message와 유사한 별도의 User-defined protocol을 사용
2. MCU에서 user-defined protocol을 수신하고 해석
3. Micom에서 CAN 통신 포트 설정 (init, open 등)
4. 전송할 항목을 CAN 통신 프로토콜로 전송

이번 실습은 CAN 통신 실습을 위해 TOPST D3-G 보드를 사용하며 코어당 수행하는 기능을 **통합하여 데모**

- A72 core : IPC Task 수행
- R5 core : CAN Controller 수행

# D3-G CAN 실습 사전 조건 - IPC TASK 이해하기



D3-G chip : Dolphin3

- IPC : Inter Processor Communication 의 약자로, **CPU와 MCU 간의 통신을 지원하기 위한 통신 구조**

Q) CAN 드라이버를 CPU에서 직접 안 쓰나요?

➔ A72는 기존 IVI를 담당하는 코어, A72가 CAN 까지 관리한다면, 코어가 예기치 못한 오류로 죽을때, 차량 제어 영역이 매우 critical해짐.

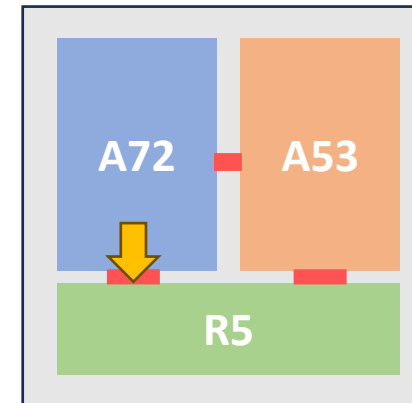
Q) 그래서 CPU가 CAN을 사용하기 위해선?

➔ CPU > IPC > MCU > CAN 드라이버 > 차량 네트워크

쉽게 설명하여, IPC 는 CPU가 MCU에게 “이 메시지를 좀 보내줘” 라고 전달하는 우체부 역할

# D3-G CAN 실습 사전 조건 - IPC TASK 이해하기

- IPC\_Example.py의 전체 동작 구조는 아래와 같음
1. A72 쪽 사용자 입력 (IPC\_Example.py)
    - argparse로 모드와 데이터를 입력 받음 (snd /rev / sndrecv)
    - snd : send
    - rev : receive
    - sndrecv : send and receive
  2. 패킷 구성 및 송신 요청
    - IPC\_SendPacketWithIPCHeader() 호출
    - 데이터 -> 헤더 + 명령 + CRC 구조로 패킹
    - /dev/tcc\_ipc\_micom 파일로 전송
  3. R5 쪽 수신 및 응답
    - R5 가 패킷을 수신 -> CAN 드라이버로 전달 (rev 모드에서는 응답 패킷 수신)



D3-G chip : Dolphin3

# D3-G CAN 실습 사전 조건 - IPC TASK 이해하기

## • IPC\_Example.py 내 main()

```
def main():
    parser = argparse.ArgumentParser(description="IPC Sender/Receiver")
    parser.add_argument("mode", choices=["snd", "rev", "sndrecv"], help="'snd': send, 'rev': receive, 'sndrecv': send then receive once")
    parser.add_argument("--file_path", default="sndfile", help="File path for IPC communication") # sndfile = open("/dev/tcc_ipc_micom", 'wb')
    parser.add_argument("--channel", type=int, default=1, help="can channel bit flag: b111")
    parser.add_argument("--txOnly", type=int, default=0, help="tx only channel bit flag: b111 (dec: 1, 2, 4) ")
    parser.add_argument("--canID", type=parse_hex16, default=0, help="CAN ID max 16-bit: e.g. 123 or 0x1234")
    parser.add_argument("--sndDataHex", type=str, help="Value for sndData as a hex string, e.g., '12345678' (default: string)")
    parser.add_argument("--sndData", type=str, default="12345678", help="Default data if no sndDataHex provided")

    ...

    sndDataLength = len(sndData)
    channel_str = ", ".join(parse_channels(channel_bitmask))
    txonly_str = ", ".join(parse_channels(tx_only_channel_bitmask))
    print(f"channel_bitmask: 0x{channel_bitmask:02X} (channels: {channel_str or 'None'})")
    print(f"tx_only_channel_bitmask: 0x{tx_only_channel_bitmask:02X} (channels: {txonly_str or 'None'})")
    print(f"canID: 0x{canID:04X}")
    print(f"sndData: {sndData}")
    print(f"sndDataLength: {sndDataLength}")
    ret = IPC_SendPacketWithIPCHheader(args.file_path, channel_bitmask, tx_only_channel_bitmask, canID, sndData)
```

- --file\_path 는 /dev/tcc\_ipc\_micom 을 기본으로 사용
- IPC\_SendPacketwithIPCHheader 함수를 통해, canID, sndData 전송에 중점
- 전송모드 snd, recv, sndrecv 명시 가능하나,
- --channel : CAN 통신 채널을 의미하며 1, 2, 4는 각각 0번, 1번, 2번 채널을 의미
- --canID : CAN 노드의 Identifier 를 의미
- --sndDataHex : 전송하고자 하는 전체 data를 의미하며 HEX 값으로 2자리씩 붙여서 명시한다 (ex : 0x123456 를 전송할때 '123456' 로 명시)

# D3-G CAN 실습 사전 조건 - IPC TASK 이해하기

- IPC\_SendPacketWithIPCHheader()의 구조는 아래와 같음
- function definition

IPC\_SendPacketWithIPCHheader(sndFile, channel\_bitmask, tx\_only\_channel\_bitmask, canID, pucData)

- function flow

1. 빈 패킷 생성 (array)
2. 헤더 작성 (SYNC, START1, START2)
3. 길이 설정 (LENGTH)
4. CM3 추가 ( Channel, txOnly, CAN ID)
5. 데이터 삽입(pucData)
6. CRC 계산 후 붙이기
7. bytes로 변환
8. /dev/tcc\_ipc\_micom에 write()

```
packet_send = array.array('B', [0] * IPC_MAX_PACKET_SIZE)
...
cnt = 0
for cnt in range(pucData_len):
    packet_send[IPC_PACKET_PREPARE_SIZE + uiCmd3_size + cnt] = pucData[cnt]
...
packet_bytes = bytes(packet_send[:total_size])
...
while True:
    try:
        sndFile.write(packet_bytes)
        now = time.time() * 1000
        sndFile.flush() #
    ...
```

# A72 IPC 세팅 & 실행

1. 1일차에 FWDN 한 ubuntu d3-g를 mobaXTerm으로 연결한다. (id : TOPST / passwd : TOPST)
2. 아래와 같은 커맨드를 입력해서, tcc\_ipc\_micom 가 설정되어 있는지 확인한다.

```
topst@TOPST:~$ ls -al /dev/tcc_ipc*
crw----- 1 root root 244, 0 Jun  4 14:17 /dev/tcc_ipc_ap
crw----- 1 root root 245, 0 Jun  4 14:17 /dev/tcc_ipc_micom
topst@TOPST:~$
```

3. 아래와 같이 실습용 확인 (D3-G 보드)

```
$ cd topst_3day_course
$ ls
```

```
topst@TOPST:~/topst_3day_course$ ls
data_parsing.py  IPC_Example.py  Library
```

- IPC\_Example.py 사용예정
  - IPC\_Example.py - MCU로 메시지 송수신하는 IPC 코드 예제
  - IPC\_Library.py - MCU로 메시지 송수신하는 IPC 라이브러리

```
├─ IPC_Example.py
└─ Library
   └─ IPC_Library.py
```

# A72 IPC 세팅 & 실행

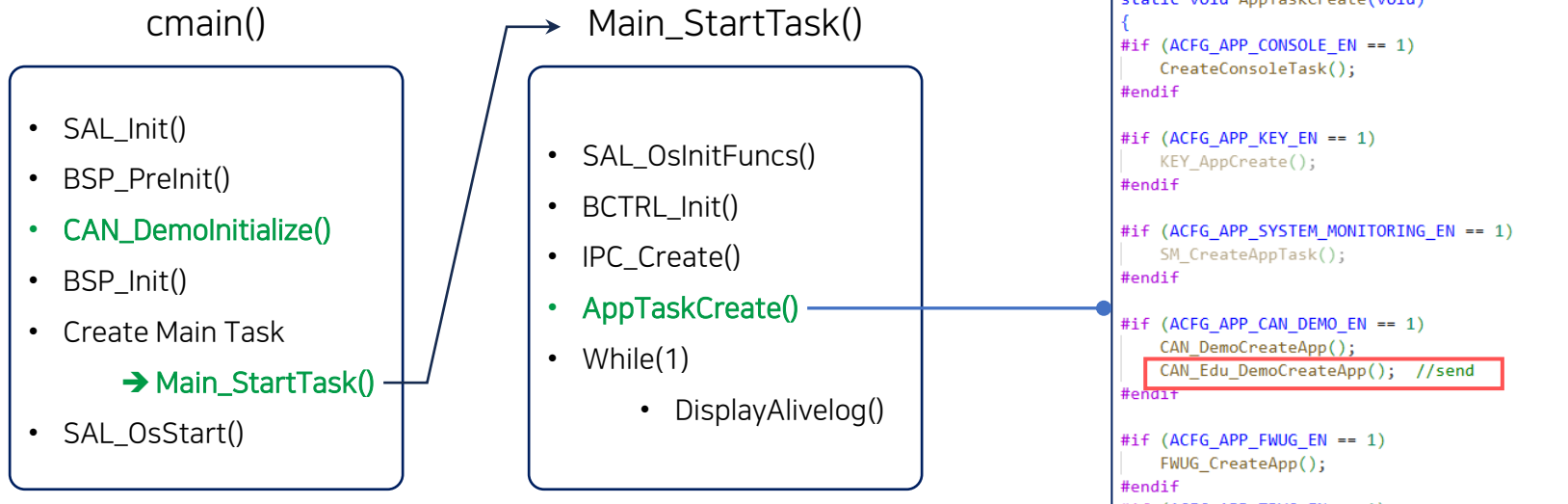
## • IPC 전송 예제 실행

- 시리얼 통신으로 연결된 Putty 로그를 통해 A72에서 전송한 메시지를 아래 커맨드로 확인할 수 있다.
- `$ sudo python3 ./IPC_Example.py snd --sndData 1122`

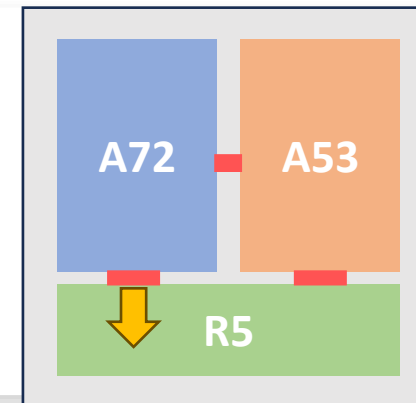
```
topst@TOPST:~/Education/d3-g app$ sudo python3 IPC_Example.py snd --sndData 1122
args.mode: snd, args.channel: 1, args.txOnly: 0, args.canID: 0x0000
channel_bitmask: 0x01 (channels: 0)
tx_only_channel_bitmask: 0x00 (channels: None)
canID: 0x0000
sndData: b'1122'
sndDataHex: 31313232
sndDataLength: 4
example : python3 IPC_Example.py sndrecv --sndData [data]
topst@TOPST:~/Education/d3-g app$
```

- CAN Channel , ID 는 default 값 사용
- Data 1122를 송신
- 출력되는 결과 : byte 1122 정상 출력완료
- 송신 데이터 길이 : 4 정상 출력완료

# D3-G CAN 실습 사전 조건 - R5 CAN Task 이해하기



- D3-G R5 내에는 FreeRTOS 세팅
- 실질적으로 CAN TASK가 이루어지는 코어
- CAN\_Edu\_DemoCreateApp() 실행



# R5 core : CAN Task 이해하기

```
static void CAN_Edu_DemoSend
(
    uint8          ucCh
)
{
    CANMessage_t    canTxMsg;

    ...

    CANDemoData_t*  canData = &gDemoData[ ucCh ];

    ...

    // CAN 프레임 속성 결정
    canTxMsg.mBufferType    = CAN_TX_BUFFER_TYPE_QUEUE;
    canTxMsg.mExtendedId    = (canData->value.id > CAN_DEMO_ID_MAX) ? TRUE : FALSE;
    canTxMsg.mId            = canData->value.id;
    canTxMsg.mFDFormat      = gNowMetaInfo.isFD;
    canTxMsg.mDataLength    = canData->value.length;
    SAL_MemCopy( canTxMsg.mData, canData->value.data, canTxMsg.mDataLength ); //CAN 데이터 구성

    CANErrorType_t ret = CAN_SendMessage( ucCh, &canTxMsg, &canTxBufferIndex ); // CAN 메시지 전송 요청

    ...

    ...

}
```

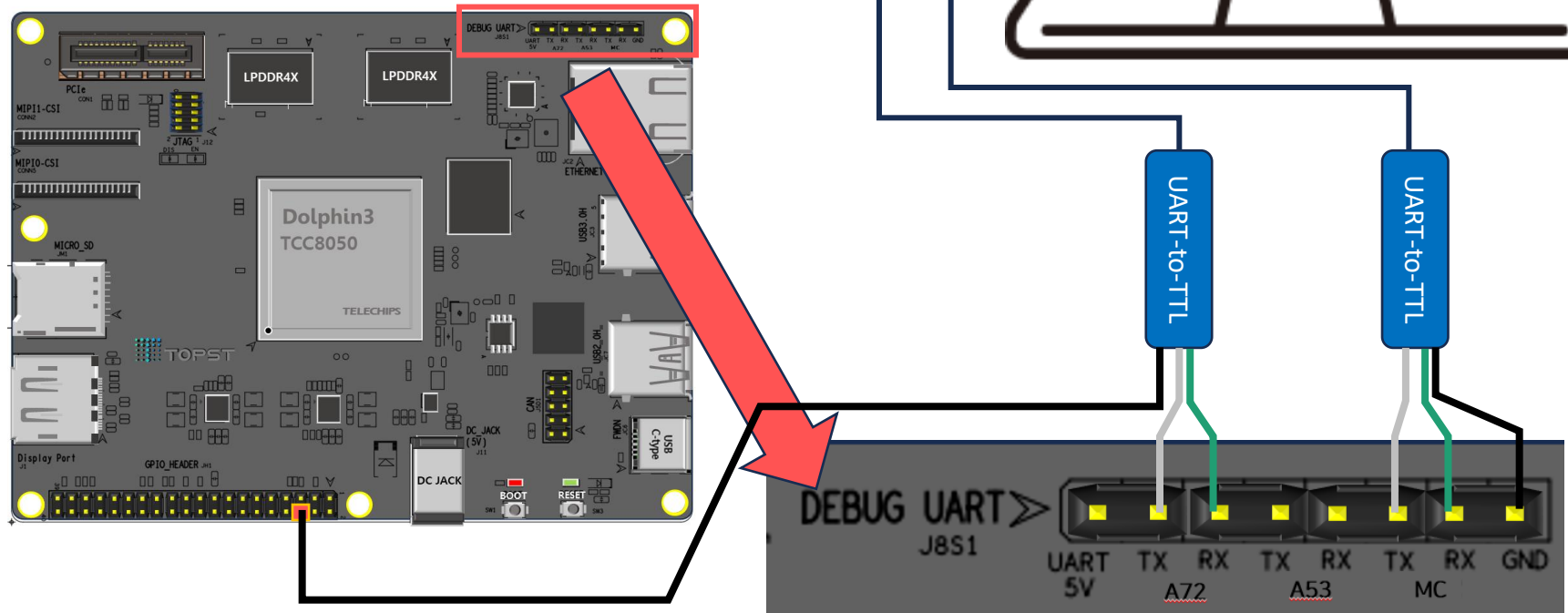
```
typedef struct CANMessage
{
    CANMessageBufferType_t    mBufferType;
    uint8                     mBufferIndex;
    uint8                     mErrorStateIndicator;
    uint8                     mExtendedId;
    uint8                     mRemoteTransmitRequest;
    uint32                    mId;
    uint8                     mFDFormat;
    uint8                     mBitRateSwitching;
    uint8                     mMessageMarker;
    uint8                     mEventFIFOControl;
    uint8                     mDataLength; //byte unit
    uint8                     mData[CAN_DATA_LENGTH_SIZE];
} CANMessage_t;
```

//CAN 송신 데이터 참조

- CAN 프레임을 구성하고 전송하는 핵심 송신 루틴

# D3-G 보드 디버깅 환경 세팅

UART-to-TTL serial 케이블 2대를 동시에 사용하여  
터미널 프로그램으로 각각 A72 / R5 코어 통신



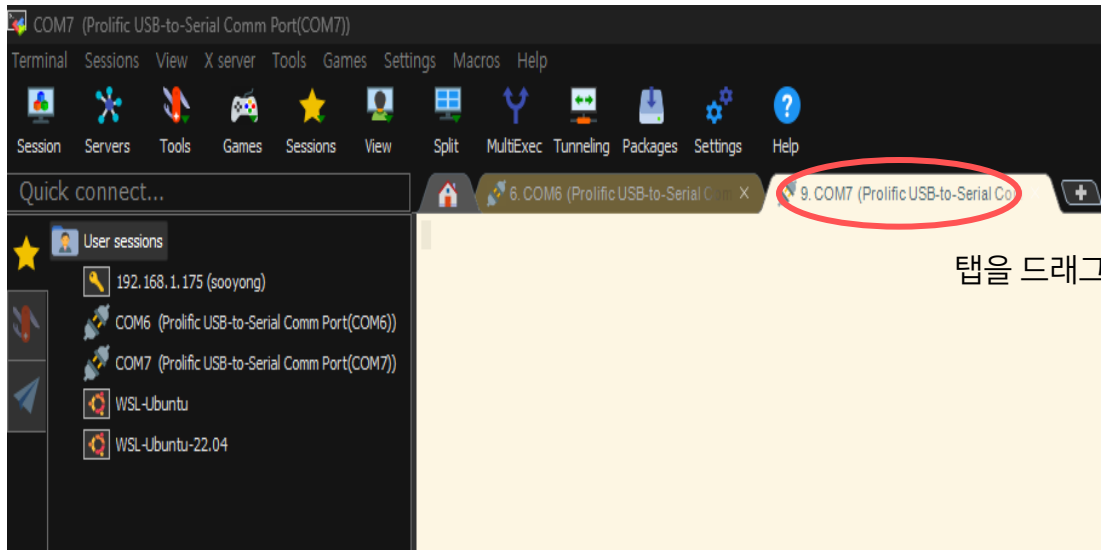
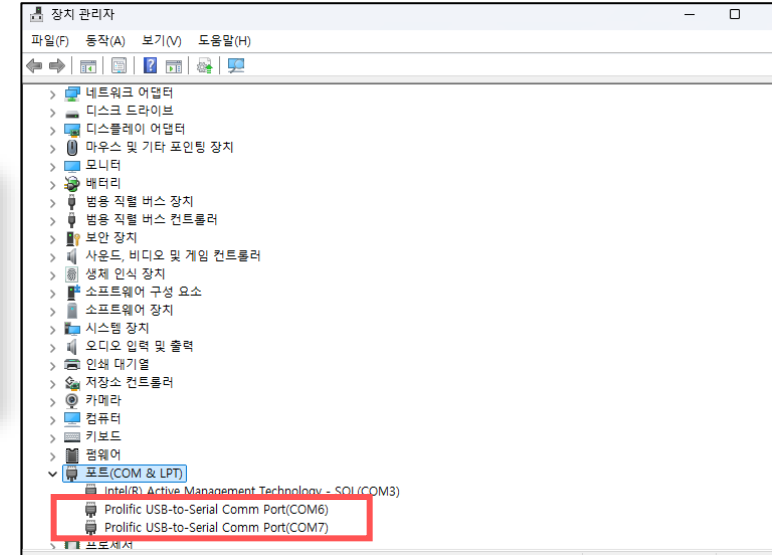
반드시 6번Pin - GND 연결  
연결 시, 점퍼선 활용

# D3-G 보드 디버깅 환경 세팅

mobaXTerm Session > Serial 2개 세팅 (포트번호는 사용자 PC마다 다름)

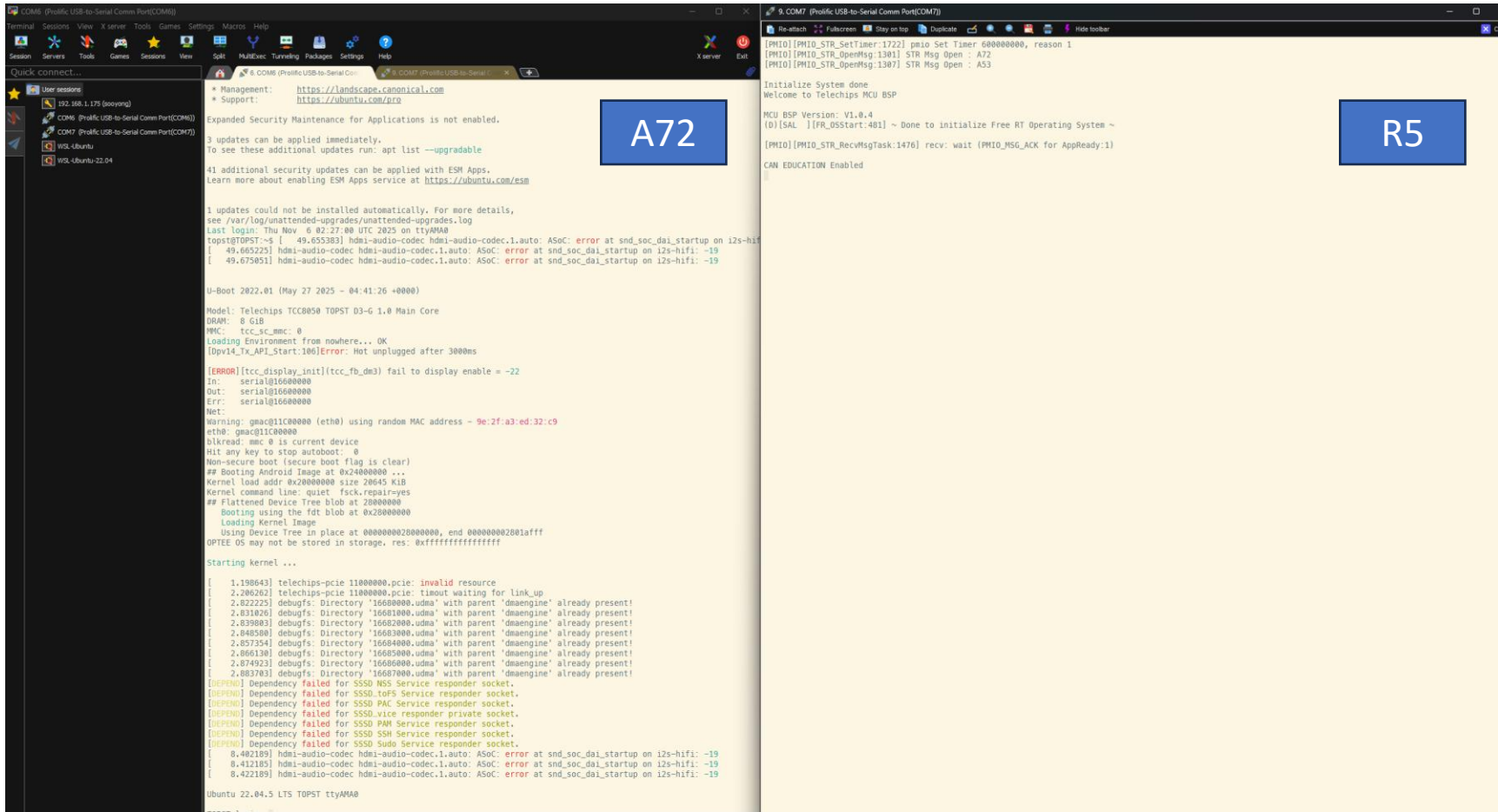
장치 관리자에서 확인

- COM 6 – A72
- COM 7 – R5



탭을 드래그해서 다른 화면에 새로 놓을 수 있음

# D3-G 보드 디버깅 환경 세팅



# D3-G CAN 통합 데모 시나리오

데모 순서는 아래와 같음

1. R5 부팅후, 아래 커맨드 입력

➤ **log on**

2. A72 에서 IPC 파이썬 스크립트 실행

```
$ cd topst_3day_course
```

```
$ sudo python3 IPC_Example.py snd --channel 1 --canID 001 --sndDataHex 00112233
```

```
ch 0 : 1 (001)  001 > 0x001 [hex]      00112233 > 0x00112233
ch 1 : 2 (010)
ch 2 : 4 (100)
```

## A72 화면

```
topst@TOPST:~/Education/d3-g app$ sudo python3 IPC_Example.py snd --channel 1 --canID 001 --sndDataHex 00112233
args.mode: snd, args.channel: 1, args.txOnly: 0, args.canID: 0x0001
channel_bitmask: 0x01 (channels: 0)
tx_only_channel_bitmask: 0x00 (channels: None)
canID: 0x0001
sndData: b'\x00\x11"3'
sndDataHex: 00112233
sndDataLength: 4
example : python3 IPC_Example.py sndrecv --sndData [data]
topst@TOPST:~/Education/d3-g app$
```

# D3-G CAN 통합 데모 시나리오

## A72 화면

```
topst@TOPST:~/Education/d3-g app$ sudo python3 IPC_Example.py snd --channel 1 --canID 001 --sndDataHex 00112233
args.mode: snd, args.channel: 1, args.txOnly: 0, args.canID: 0x0001
channel_bitmask: 0x01 (channels: 0)
tx_only_channel_bitmask: 0x00 (channels: None)
canID: 0x0001
sndData: b'\x00\x11"3'
sndDataHex: 00112233
sndDataLength: 4
example : python3 IPC_Example.py sndrecv --sndData [data]
topst@TOPST:~/Education/d3-g app$
```

## R5 화면

```
(D) [CAN ] [CAN_Edu_DemoSend:1353] *****
(D) [CAN ] [CAN_Edu_DemoSend:1354] [ID] : 0x1, [DATA SIZE] : 4, [DATA] :
(D) [CAN ] [CAN_Edu_DemoSend:1357] 0x00
(D) [CAN ] [CAN_Edu_DemoSend:1357] 0x11
(D) [CAN ] [CAN_Edu_DemoSend:1357] 0x22
(D) [CAN ] [CAN_Edu_DemoSend:1357] 0x33
(D) [CAN ] [CAN_Edu_DemoSend:1363]
(D) [CAN ] [CAN_Edu_DemoSend:1364] *****
```

# D3-G CAN 통합 데모 시나리오

- 실습용 CAN message 정의 (Ch : 0)

| 메시지 종류   | ID    | Data                    |
|----------|-------|-------------------------|
| CLASS_00 | 0x100 | [0] : on-0x01, off-0x02 |
| CLASS_01 | 0x101 | [0] : on-0x01, off-0x02 |
| CLASS_02 | 0x102 | [0] : on-0x01, off-0x02 |
| CLASS_03 | 0x103 | [0] : on-0x01, off-0x02 |

# D3-G CAN 통합 데모 시나리오

데모 순서는 아래와 같음

1. R5 부팅후, 아래 커맨드 입력  
➤ log on
2. 메시지 포맷에 맞춰서 IPC 스크립트 실행

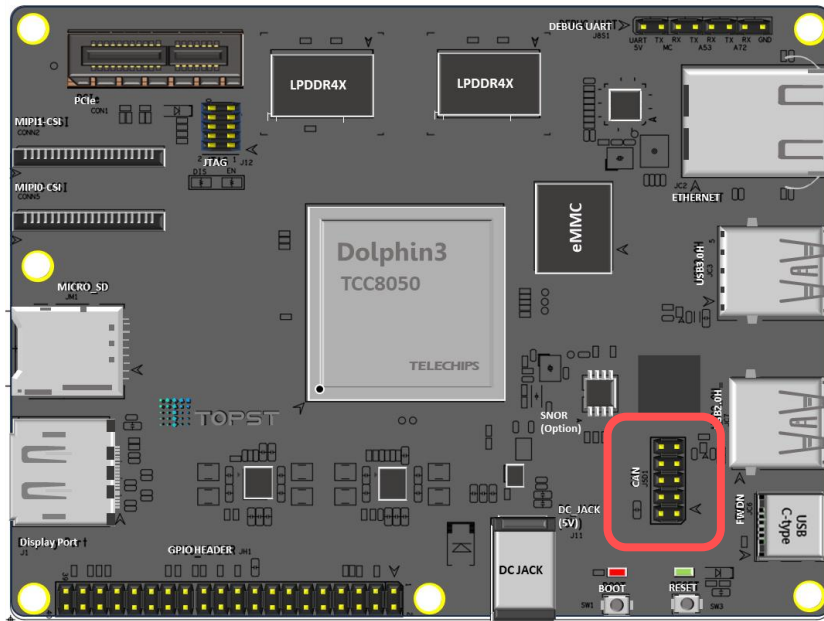
```
$ cd ~/topst_3day_course
$ sudo python3 IPC_Example.py snd --channel 1 --canID 100 --sndDataHex 01
$ sudo python3 IPC_Example.py snd --channel 1 --canID 101 --sndDataHex 01
$ sudo python3 IPC_Example.py snd --channel 1 --canID 102 --sndDataHex 01
$ sudo python3 IPC_Example.py snd --channel 1 --canID 103 --sndDataHex 01
```

| 메시지 종류   | ID    | Data                    |
|----------|-------|-------------------------|
| CLASS_00 | 0x100 | [0] : on-0x01, off-0x02 |
| CLASS_01 | 0x101 | [0] : on-0x01, off-0x02 |
| CLASS_02 | 0x102 | [0] : on-0x01, off-0x02 |
| CLASS_03 | 0x103 | [0] : on-0x01, off-0x02 |

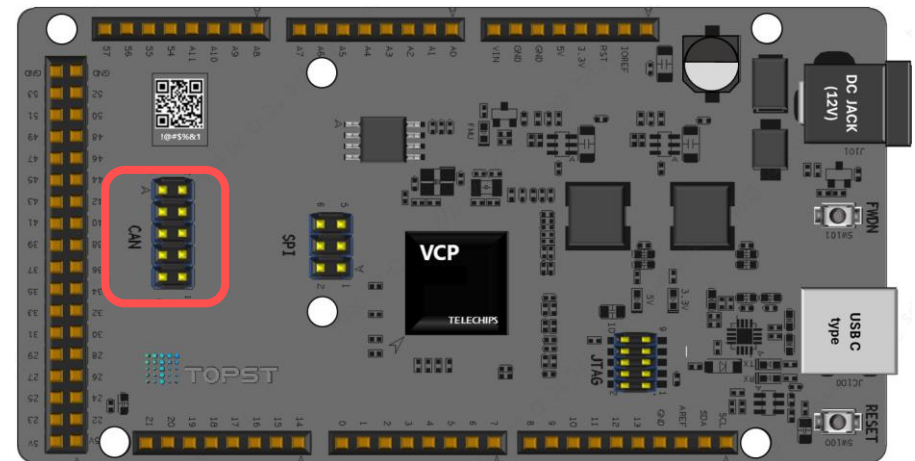
➤ 커맨드 입력하고 R5 터미널에서 결과 확인하기

# Support to CAN pin

- Q) TOPST 보드는 어떤 형태로 CAN을 지원하는지
- TOPST D3-G CAN 지원은 3ch까지 가능 - 10개의 PIN 형태로 지원
- TOPST VCP-G CAN 지원은 3ch까지 가능 - 10개의 PIN 형태로 지원



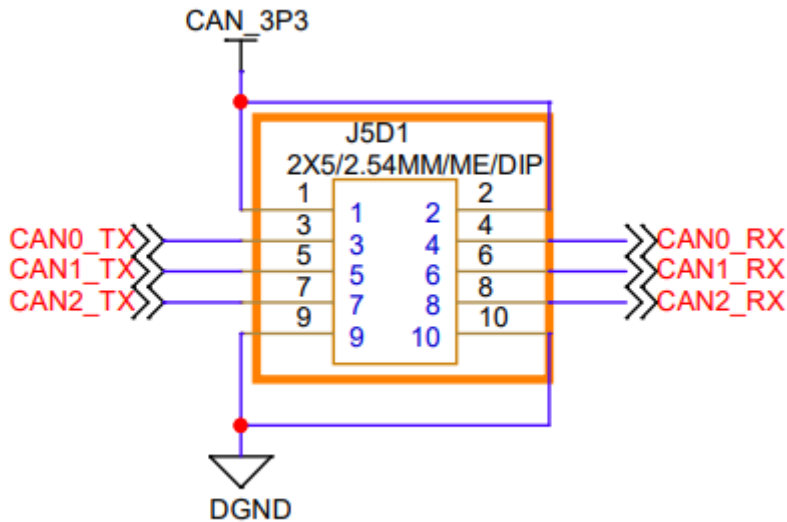
D3-G



VCP-G

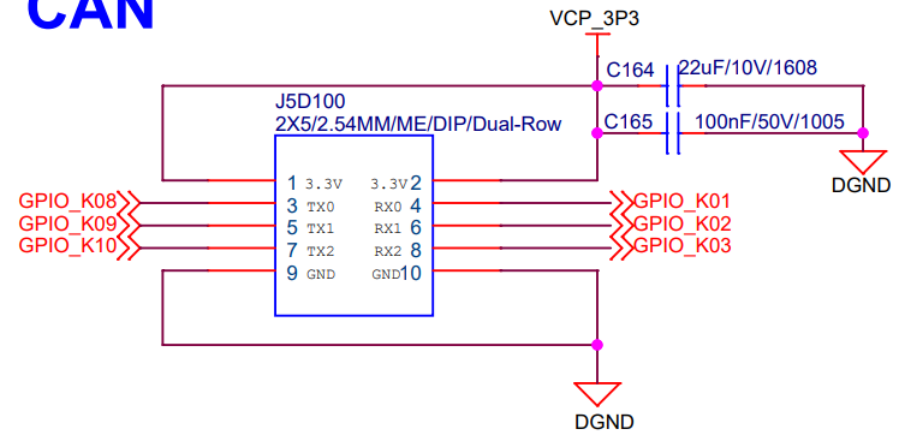
# Support to CAN pin

- Q) TOPST 보드는 어떤 형태로 CAN을 지원하는지
- TOPST D3-G CAN 지원은 3ch까지 가능 - 10개의 PIN 형태로 지원
- TOPST VCP-G CAN 지원은 3ch까지 가능 - 10개의 PIN 형태로 지원
- 핀의 형태는 두 보드 동일



D3-G CAN Port

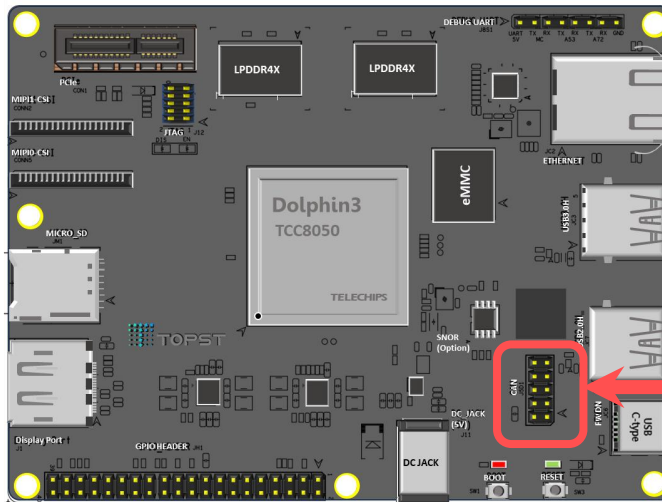
## CAN



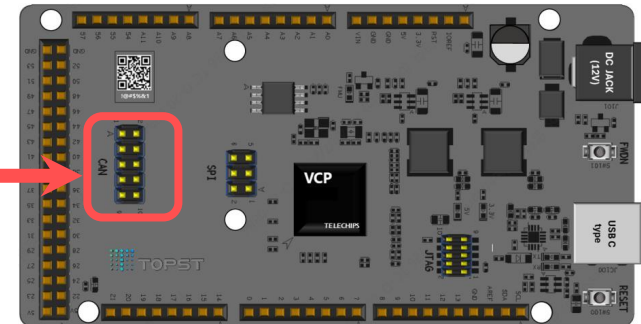
VCP-G CAN Port

# Support to CAN pin

- Q) 물리적으로 서로 연결하기만 한다면 CAN통신이 되나요?
  - CAN 버스는 차동 신호(CAN High, CAN Low)로 통신하기 때문에 단순히 TX, RX 방식의 통신을 할 수 없음
- D3-G와 VCP-G의 CAN 통신을 위한 CAN Transceiver 장비를 사용
- 본 과정에서 SN65HVD230 VP230 CAN 모듈 사용 예정



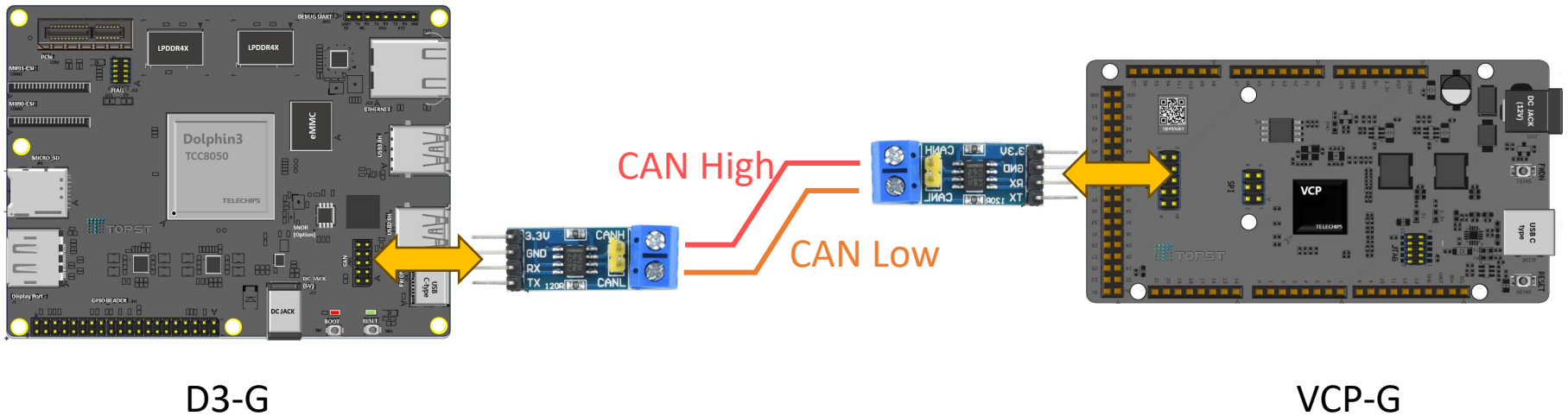
D3-G



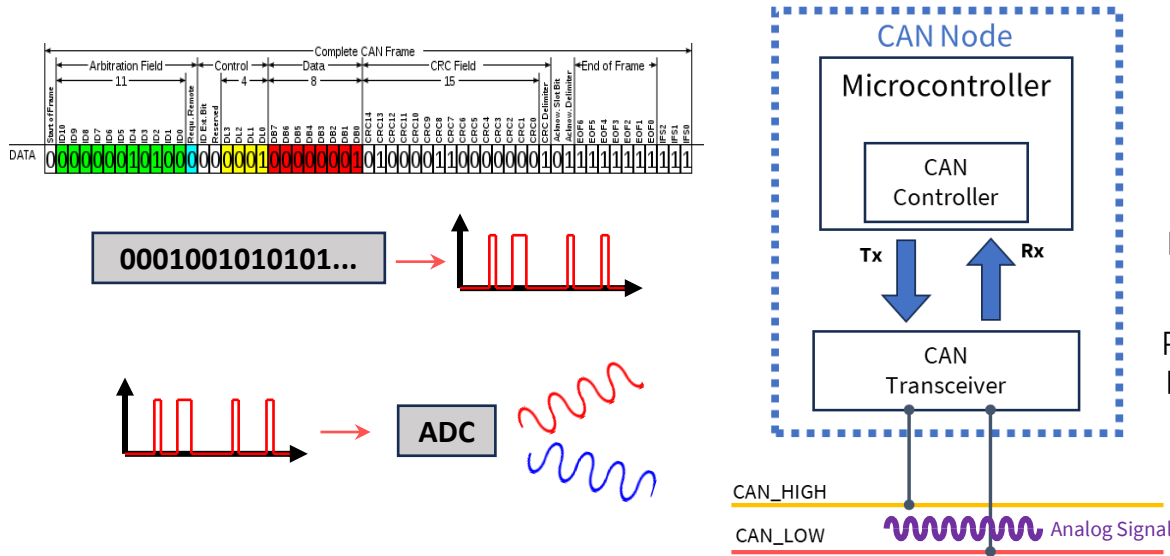
VCP-G

# Support to CAN pin

- 따라서, 별도의 CAN Transceiver 장비를 사용해야, 보드간 CAN 통신을 수행할 수 있다.



# CAN Message 전송 흐름



Data Link Layer  
ISO 11898-1

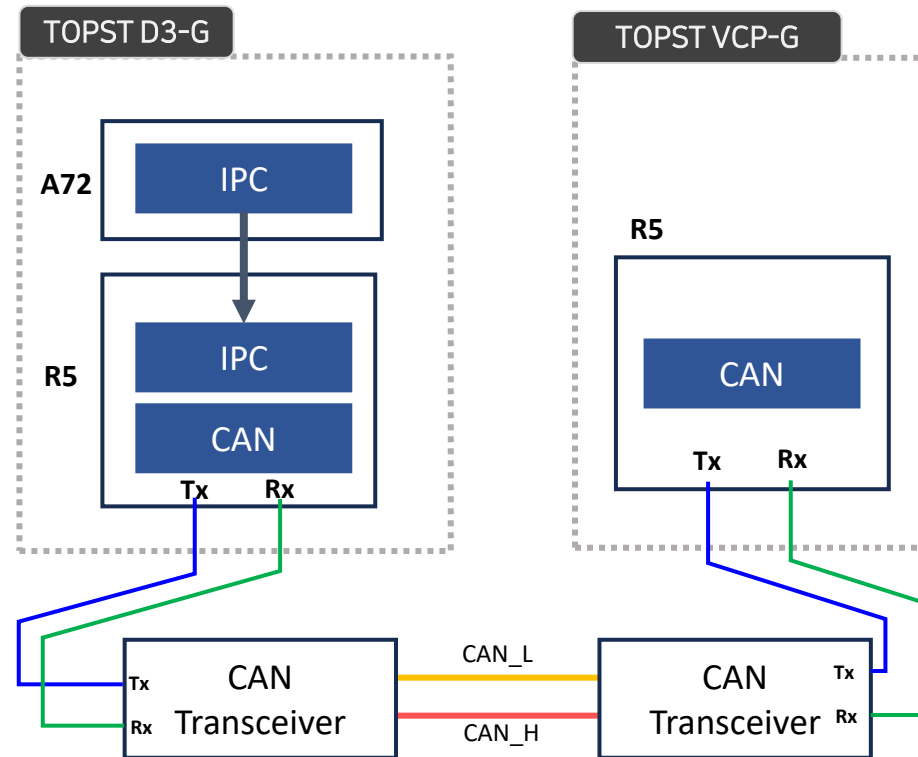
Physical Layer  
ISO 11898-2/  
11898-3

**ISO 11898-1:** CAN 데이터 링크 계층을 정의한다. CAN 프로토콜에서 데이터의 전송 및 오류 처리 기능 뿐만 아니라 표준 CAN 메시지의 구조와 프레임의 형식을 규정한다.

**ISO 11898-2:** 고속 CAN(CAN High-Speed)의 물리 계층을 정의한다. 차량 내 통신에서 주로 사용되며, 데이터 전송 속도는 최대 1 Mbps이다.  
**ISO 11898-3:** 저속 CAN(CAN Low-Speed)의 물리 계층을 정의한다. 주로 차량의 간단한 네트워크 및 저속 통신에 사용되며 Fault Tolerant(내고장성, 결함 허용) 특징을 가지고 있다.

- CAN 통신과 연관된 동작을 하는 장치 : CAN controller
  - CAN controller 는 MCU 내부에 마련되어 있음
  - CAN 통신의 프로토콜을 따라서 실제로 CAN 통신을 할 수 있도록 지원
- CAN 컨트롤러에서 링크 계층 표준을 만족하며, 데이터의 전송, 수신 및 프로토콜을 담당한다.
- CAN Transceiver( **T**ransmitter + **R**eciever)
  - 디지털 신호,아날로그 신호를 서로 변환하고 송수신하는 역할을 지원한다.
  - Controller에서 생성한 신호(메시지)를 ADC를 통해서 전압값으로 변환
  - 변환된 전압 신호(Analog Signal)을 CAN bus로 전송

# TOPST 보드간 CAN 메시지 전송 흐름

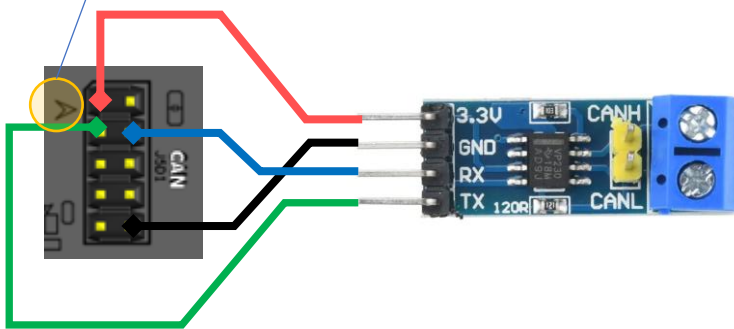


- 따라서 위와 같은 구조로 실습 수행
- D3-G [A72 >> R5] -> VCP-G 순으로 데이터 전송

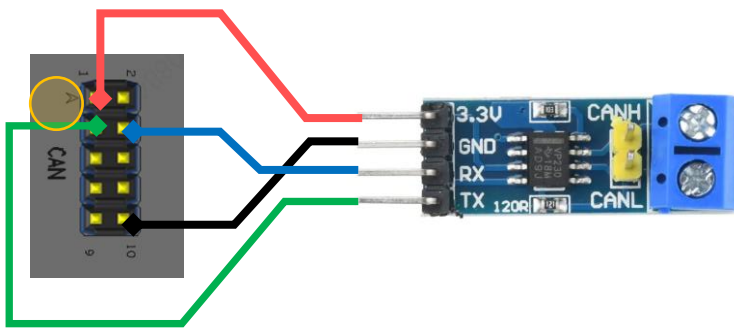
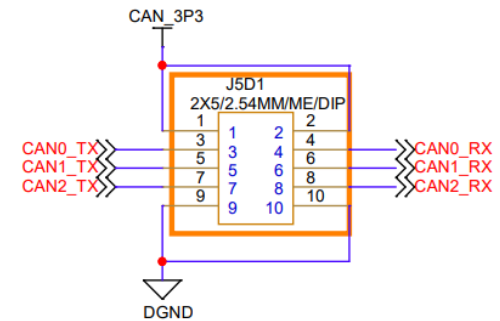
# 실습을 위한 보드 연결

두 보드간 연결을 위해 아래와 같이 회로를 결선한다.

화살표 표시 부분이 1번핀

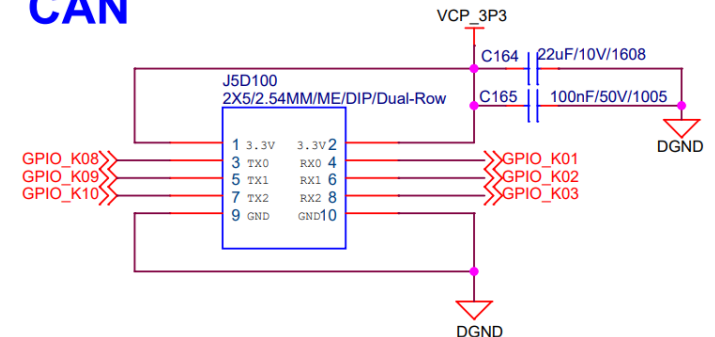


D3-G



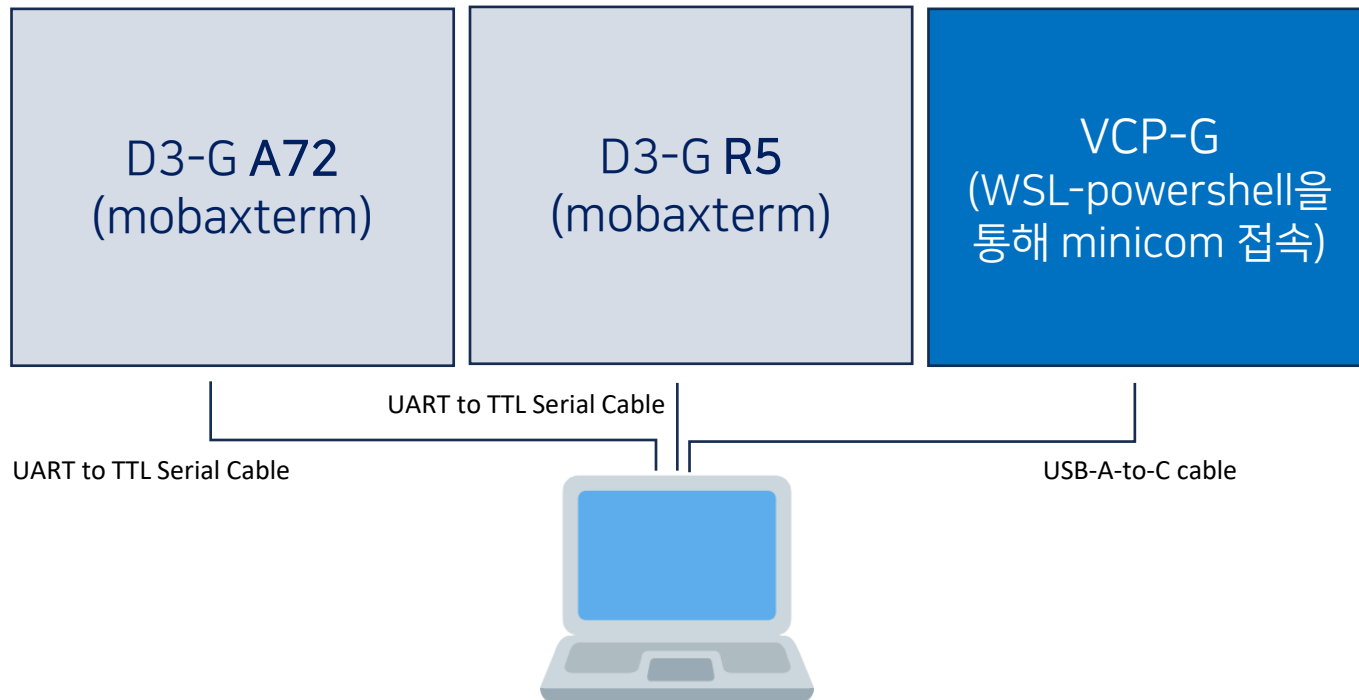
VCP-G

CAN



## 실습환경 세팅

- D3-G 보드와 VCP-G 보드와 UART 통신 세팅을 진행한다.



- A72 터미널내에서 IPC 명령어를 통해 R5 터미널과 VCP-G 터미널 화면을 관찰할 수 있다.
  - `$ cd topst_3day_course`

# 실습환경 세팅

## 1. VCP-G 실행 시 아래 커맨드 입력

```
$ can receive start
```

## 2. A72 IPC\_Example 실행 커맨드 리스트 예제

| No. | Scenario       | IPC Command                                                                                                                                             |
|-----|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | Detect class 0 | <pre>\$ sudo python3 IPC_Example.py snd --canID 0x100 --sndDataHex 01</pre> <pre>\$ sudo python3 IPC_Example.py snd --canID 0x100 --sndDataHex 02</pre> |
| 2   | Detect class 1 | <pre>\$ sudo python3 IPC_Example.py snd --canID 0x101 --sndDataHex 01</pre> <pre>\$ sudo python3 IPC_Example.py snd --canID 0x101 --sndDataHex 02</pre> |
| 3   | Detect class 2 | <pre>\$ sudo python3 IPC_Example.py snd --canID 0x102 --sndDataHex 01</pre> <pre>\$ sudo python3 IPC_Example.py snd --canID 0x102 --sndDataHex 02</pre> |
| 4   | Detect class 3 | <pre>\$ sudo python3 IPC_Example.py snd --canID 0x103 --sndDataHex 01</pre> <pre>\$ sudo python3 IPC_Example.py snd --canID 0x103 --sndDataHex 02</pre> |

# 실습 결과

```

COM8 (Prolific USB-to-Serial Comm Port(COM8))
topst@TOPST:~/topst_3day_course$ sudo python3 IPC_Example.py snd --canID 0x100 --sndDataHex 02
args.mode: snd, args.channel: 1, args.txOnly: 0, args.canID: 0x0100
channel_bitmask: 0x01 (channels: 0)
tx only channel bitmask: 0x00 (channels: None)
canID: 0x0100
sndData: b'\x02'
sndDataLength: 1
example: python3 IPC_Example.py sndrecv --sndData {data}
topst@TOPST:~/topst_3day_course$

(D)[MBOX ][MBOX_Dev0Get:1314] empty for ch[3] siDevId[2]
(D)[MBOX ][MBOX_Dev0Get:1314] empty for ch[3] siDevId[2]
(D)[CAN ][CAN_Edu_SetSendingData:1567]
ch_bitmask: 0x1, tx_only_bitmask: 0x0, can_id: 0x100
(D)[CAN ][CAN_Edu_SetSendingData:1583] [CH] : 0x0, [ID] : 0(D)[MBOX ][MBOX_Dev0Get:1314] empty for ch[3] siDevId[2]
x100, [DATA SIZE] : 1 (5), [DATA] :
(D)[CAN ][CAN_Edu_SetSendingData:1586] 0x02
(D)[CAN ][CAN_Edu_SetSendingData:1591]
(D)[CAN ][CAN_Edu_SetSendingData:1599] gNowMetaInfo.channel : 0x7
(D)[CAN ][CAN_Edu_DemoSend:1332] canTxMsg => id: 256, isFE: 0, extID: 0, len: 1, data:
(D)[CAN ][CAN_Edu_DemoSend:1345]
ret: 0
(D)[CAN ][CAN_Edu_DemoSend:1369]
CAN_SEND ch: 0 id: 256 mDataLength:1 (1:10)
(D)[CAN ][CAN_Edu_DemoSend:1375] Successful transmission.

[CAN DEMO]
< Channel 0 Received Message Information >
*****
[ID] : 0x100, [DATA SIZE] : 1, [DATA] :
0x02
*****
[LED] id=0x100 -> OFF
CH5 angle: 145° → duty: 2111095 ns
CH5 angle: 140° → duty: 2055540 ns
CH5 angle: 135° → duty: 1999985 ns
CH5 angle: 130° → duty: 1944430 ns
CH5 angle: 125° → duty: 1888875 ns
CH5 angle: 120° → duty: 1833320 ns
CH5 angle: 115° → duty: 1777765 ns
CH5 angle: 110° → duty: 1722210 ns
CH5 angle: 105° → duty: 1666655 ns
CH5 angle: 100° → duty: 1611100 ns
[CAN DEMO] No message
    
```

D3-G A72 (IPC\_Example)

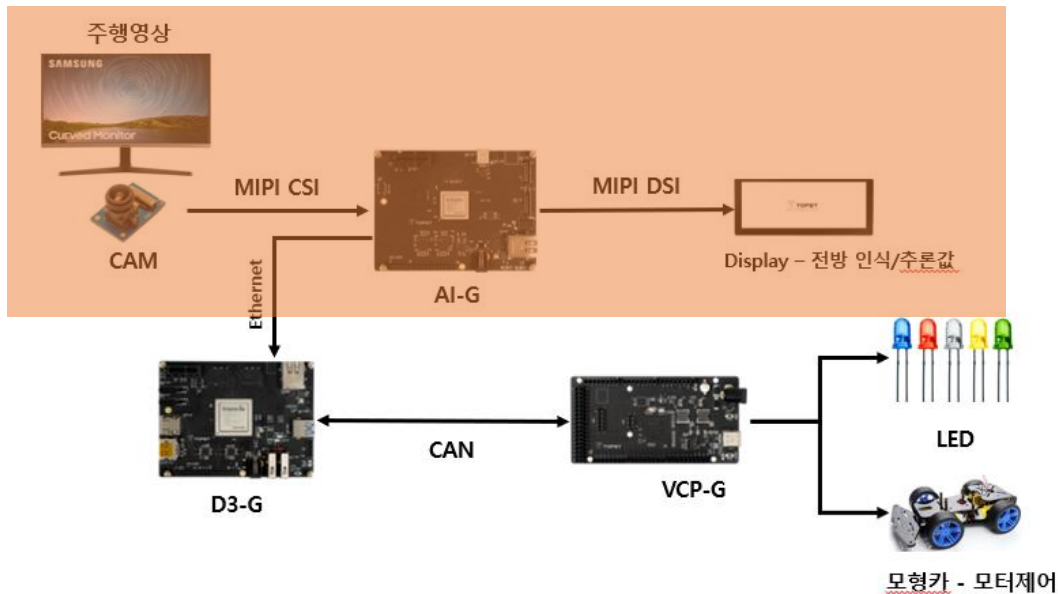


D3-G R5



VCP-G

# TOPST 통합 제어 시스템 실습



## • 동작 시나리오(AI-G)

### 1. 영상 입력

- 전방 카메라를 통해 실시간 영상 데이터 받음

### 2. 객체 인식

- NPU보드에서 YOLOv8 모델을 통해 사람, 차량 등 객체 인식 추론 수행

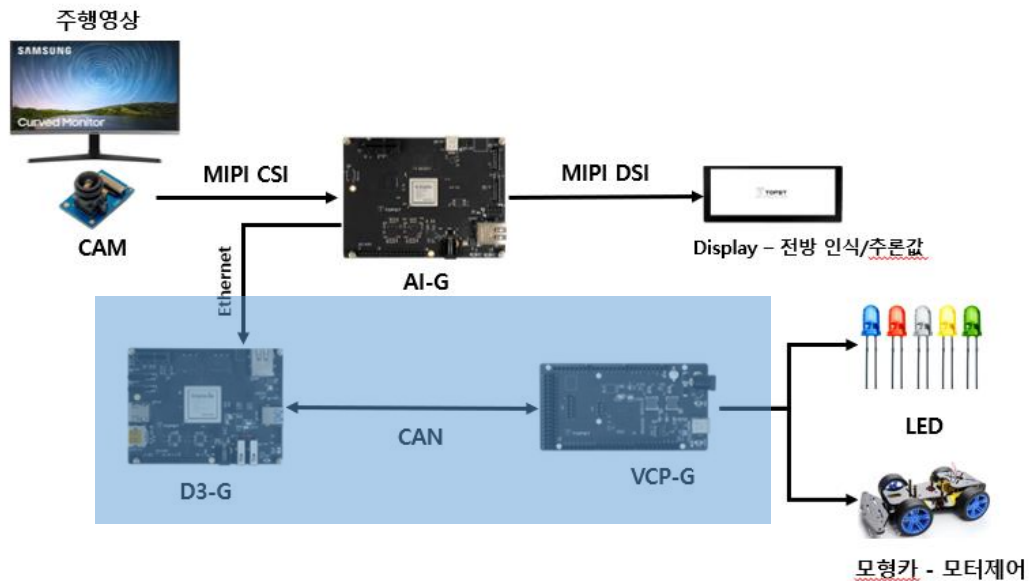
### 3. 추론 결과 시각화

- 객체 인식 결과(바운딩 박스, 클래스 정보)를 display를 통해 시각화

### 4. 추론 정보 전송

- 추론 결과 데이터를 이더넷을 통해 D3-G 보드로 전송

# TOPST 통합 제어 시스템 실습



## • 동작 시나리오(D3-G)

### 1. 추론 결과 파싱

- AI-G로 부터 전달된 추론 결과를 수신 및 파싱

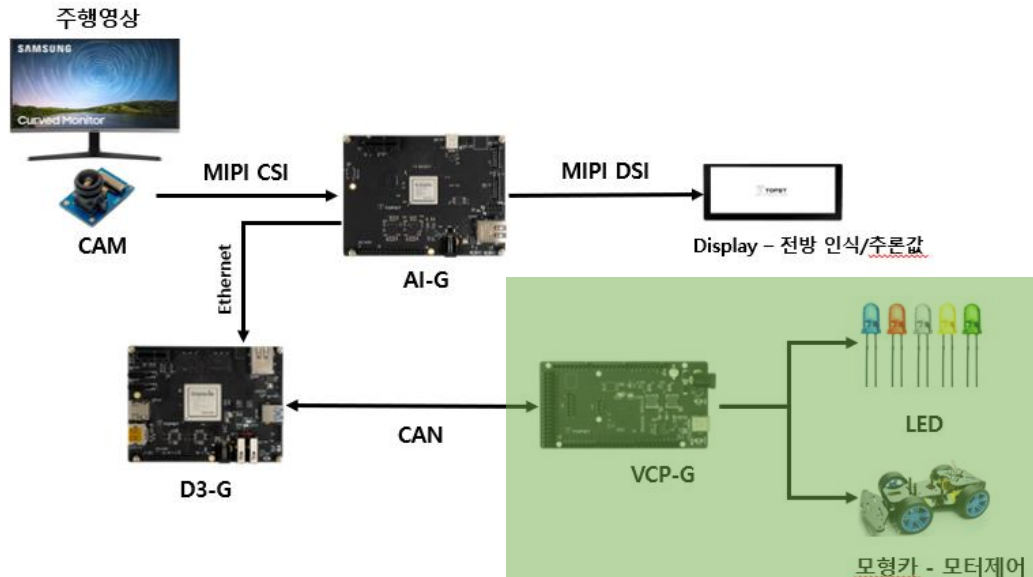
### 2. CAN 메시지 구성

- 파싱된 객체 정보를 기반으로 제어용 CAN 메시지를 생성

### 3. VCP-G로 메시지 전송

- 구성된 CAN 메시지를 CAN 통신을 통해 VCP-G로 전송

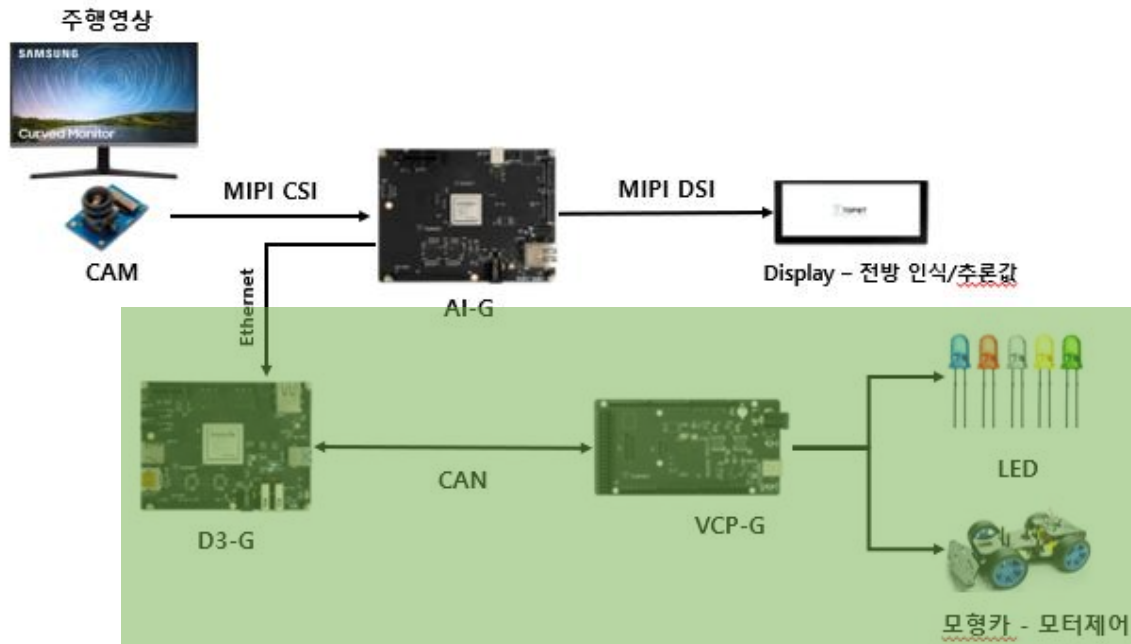
# TOPST 통합 제어 시스템 실습



## • 동작 시나리오(VCP-G)

1. 초기 주행 상태
  - 조향각 0도 상태에서 DC 모터 정속 주행 수행
2. 장애물 인식
  - 전방 장애물 인식 시 D3-G에서 차량 제어 명령 송신(CAN)
3. 경고 표시
  - 장애물 인식 메시지 송신 후 LED 제어 (경고 표시)
4. 장애물 회피 동작
  - 장애물 인식 시 조향각 조절 및 DC 모터 감속 수행
5. 정상 주행 복귀
  - 일정 시간 이후 정상 주행

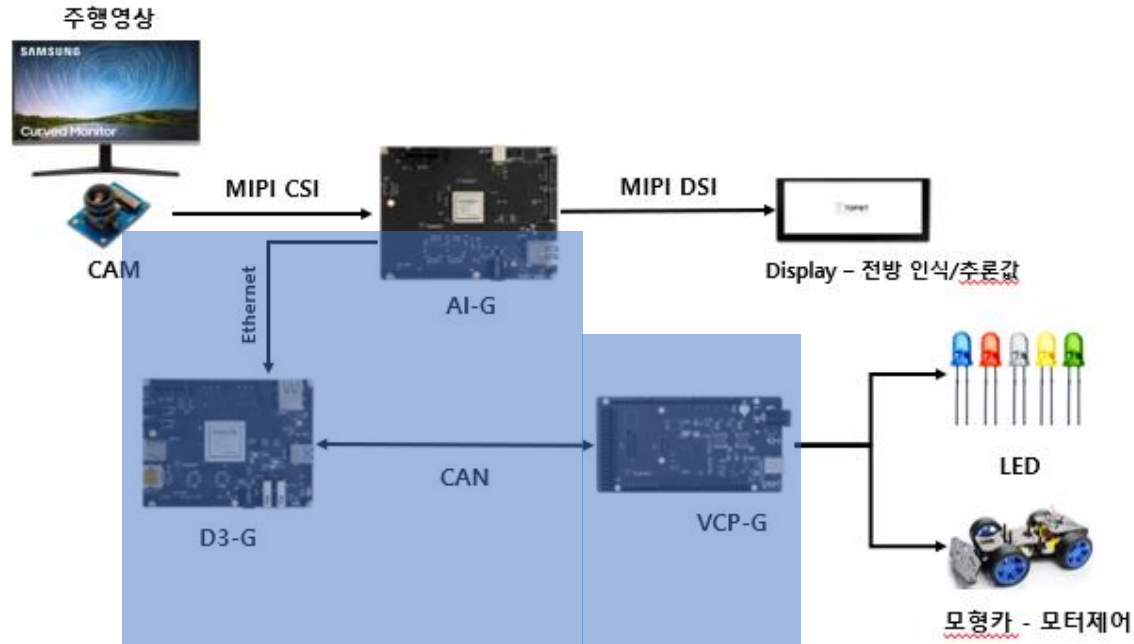
# TOPST 통합 제어 시스템 실습



## VCP-G 구성

- LED, 서모 모터, DC 모터 연결 확인
- D3-G와 CAN 모듈 연결 확인
- 12V 1A 전원 공급

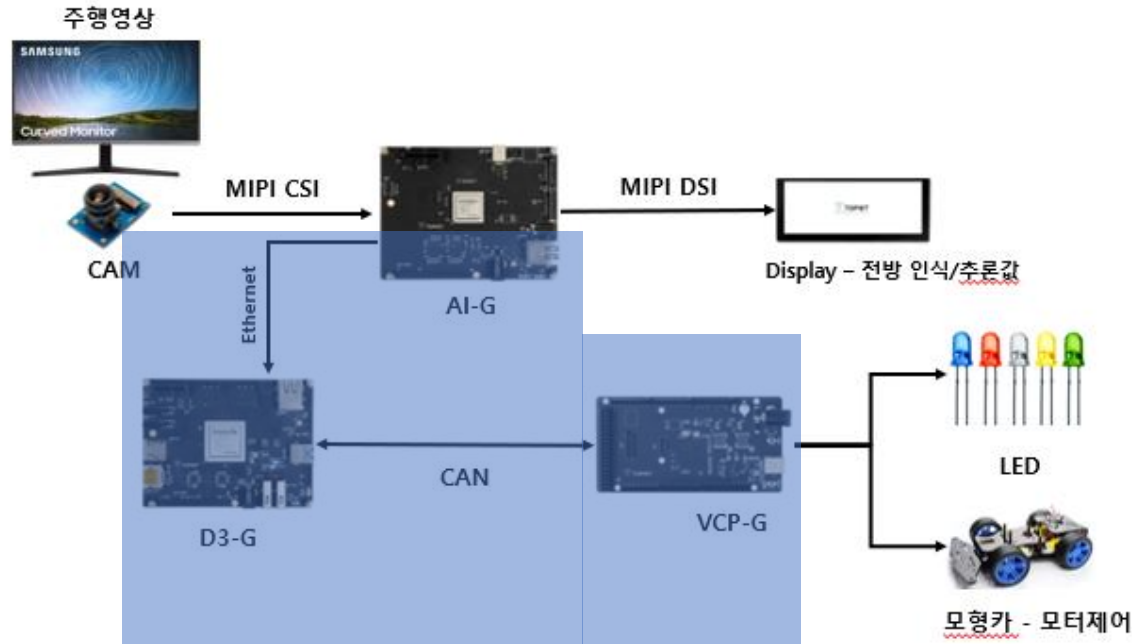
# TOPST 통합 제어 시스템 실습



## D3-G 구성

- VCP-G와 **CAN 모듈** 연결 확인
- **5V 5A 전원** 공급
- AI-G와 **이더넷 케이블** 연결
- A72에 UART TTL 케이블 연결

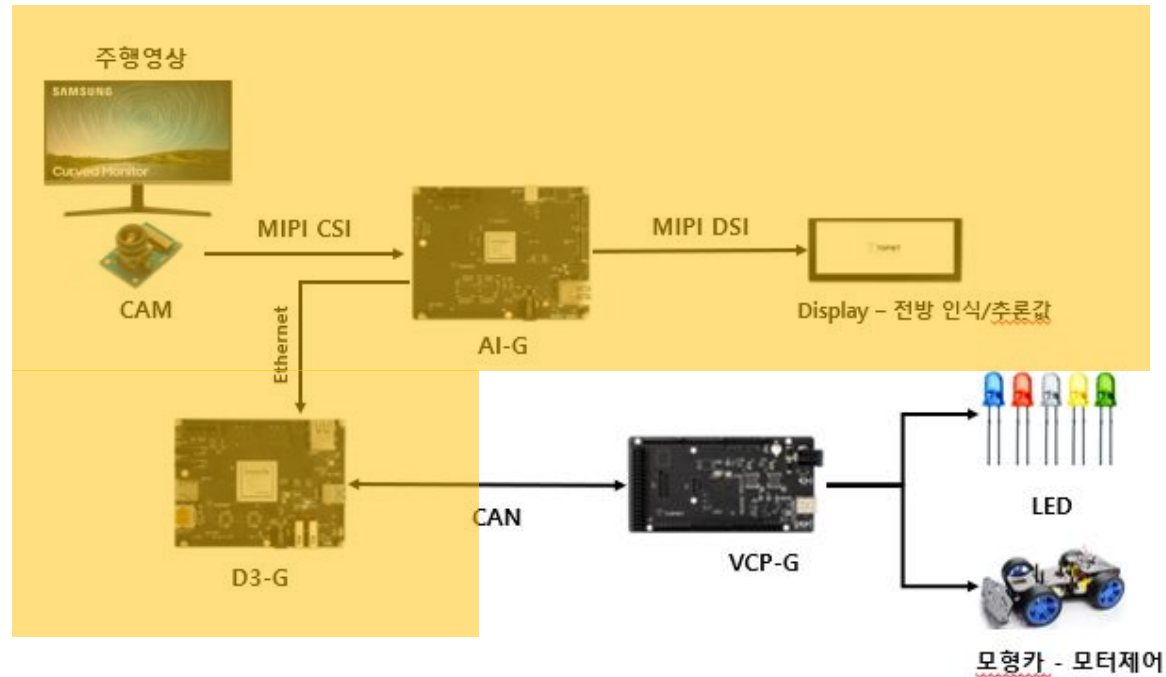
# TOPST 통합 제어 시스템 실습



## D3-G 명령어 실행

- MobaXterm에서 D3-G A72 접속 (ID : topst / PASSWD : topst) 후 로그인
- `$ cd topst_3day_course`
- `$ sudo ifconfig eth0 192.168.0.8 up`
- `$ sudo python3 data_parsing.py`

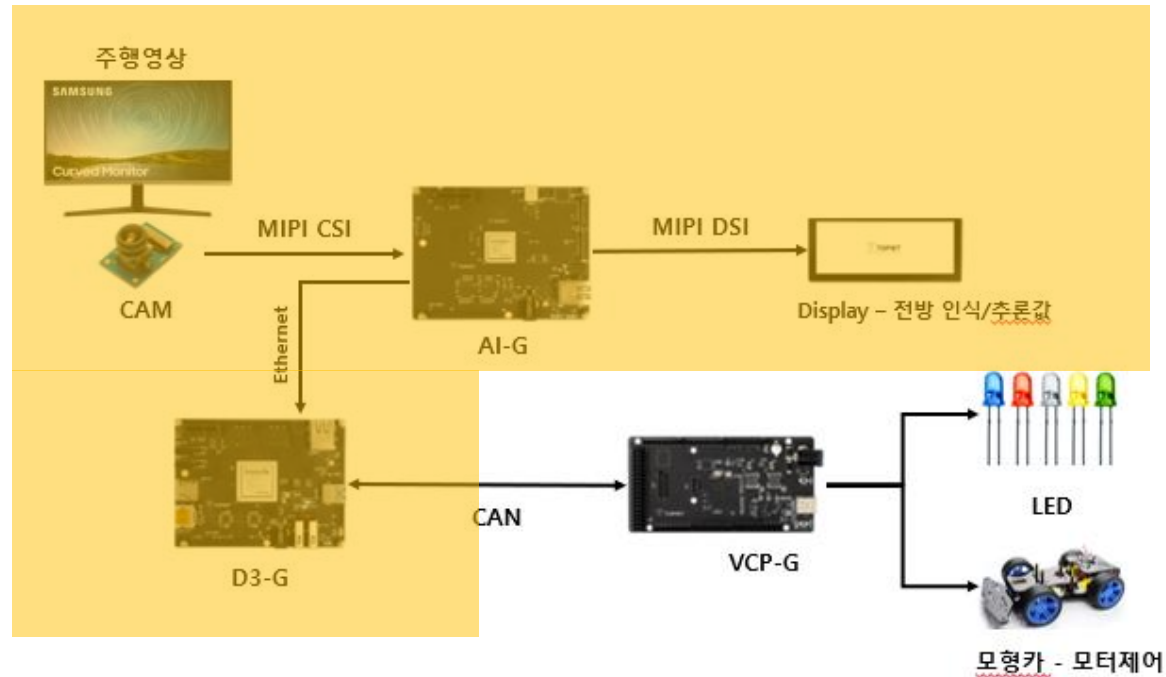
# TOPST 통합 제어 시스템 실습



## AI-G 구성

- CSI CAM, DSI DISPLAY 연결 확인
- 5V 5A 전원 공급
- D3-G와 이더넷 케이블 연결
- HOST PC와 UART TTL 케이블 연결

# TOPST 통합 제어 시스템 실습



## AI-G 명령어 실행

- `./tcnapp -n yolov8s_quantized -p /dev/video2`

# TOPST 통합 제어 시스템 코드 리뷰(D3-G)

```
import json
import time
import socket
import threading
import argparse
from dataclasses import dataclass
from typing import Dict, Any, List

from Library.IPC_Library import IPC_SendPacketWithIPCHeader
```

```
class VCP_IO:
    CLASS_00 = 0x100
    CLASS_01 = 0x101
    CLASS_02 = 0x102
    CLASS_03 = 0x103

    ACTION_ON = 0x01
    ACTION_OFF = 0x02

    # cls -> CAN ID(io_type)
    CLASS_MAP = {1: CLASS_00, 2: CLASS_01, 3: CLASS_02, 4: CLASS_03}
```

- 코드 리뷰(D3-G)

1. Library import

- 각종 통신 모듈 및 스레드 사용을 위한 모듈 설정

2. CAN Message ID 구성

- YOLO의 0~3 CLS에 해당하는 CAN 메시지 ID 구성

- 코드 경로 - topst\_3day\_course/data\_parsing.py

# TOPST 통합 제어 시스템 코드 리뷰(D3-G)

```
class Config:
    IPC_PATH: str = "/dev/tcc_ipc_micom"

    # TCP server (tc-nn-app가 client로 연결)
    LISTEN_IP: str = "0.0.0.0"
    LISTEN_PORT: int = 5000

    # IPC 송신 루프 주기
    SEND_HZ: int = 60

    # 관심 클래스/필터
    CLS_MIN: int = 1
    CLS_MAX: int = 4
    SCORE_TH: float = 0.50

    # 클래스별 디바운스(떨림 방지)
    STREAK_ON: int = 1
    STREAK_OFF: int = 5
```

- 코드 리뷰(D3-G)

- 1. 변수 설정 및 구성

- CAN 통신을 위한 IPC PATH 및 TCP/IP통신을 위한 설정, CAN 통신 주기, 디바운스 등 설정

- 코드 경로 - topst\_3day\_course/data\_parsing.py

# TOPST 통합 제어 시스템 코드 리뷰(D3-G)

```
def _send_ipc(self, io_type: int, value: int) -> bool:
    if not self.ipc_file:
        return False
    try:
        payload = int(value).to_bytes(1, "big", signed=True)
        IPC_SendPacketWithIPCHeader(self.ipc_file, 1, 0, io_type, payload)
        return True
    except Exception as e:
        print(f"[ERR] IPC Send Failed (0x{io_type:X}): {e}")
        return False
```

```
# ---- LED ON/OFF for a class ----
def _set_class_led(self, cls_id: int, on: bool):
    io_type = VCP_IO.CLASS_MAP.get(cls_id)
    if io_type is None:
        return
```

```
def _extract_best_scores(self, msg: Dict[str, Any]) -> (List[float], int):
    stamp = int(msg.get("stamp", 0))
    boxes = msg.get("boxes", []) or []
```

## • 코드 리뷰(D3-G)

### 1. CAN Message 전송

- ID와 값을 인자로 받아 CAN 메시지로 전송하는 함수

### 2. set\_class\_led()

- 특정 클래스만 구분하여 can message 전송하도록 하는 함수

### 3. extract\_best\_scores()

- 클래스 별 최고 점수만 남기고 나머지는 제거 (같은 객체를 여러 개 인식 시)

• 코드 경로 - topst\_3day\_course/data\_parsing.py

# TOPST 통합 제어 시스템 코드 리뷰(D3-G)

```
# ---- debounce & send only when state changes ----  
def _update_per_class_debounce(self, best_scores: List[float]):  
    for cls in range(self.cfg.CLS_MIN, self.cfg.CLS_MAX + 1):  
        idx = cls - self.cfg.CLS_MIN
```

```
def _worker_tcp_server(self):  
    print(f"[NET] TCP Server listening on {self.cfg.
```

```
# ---- thread 2: periodic IPC sender ----  
def _worker_ipc_sender(self):  
    print("[IPC] Sender started.")  
    period = 1.0 / max(1, self.cfg.SEND_HZ)
```

```
def start(self):  
    workers = [self._worker_tcp_server, self._worker_ipc_sender]
```

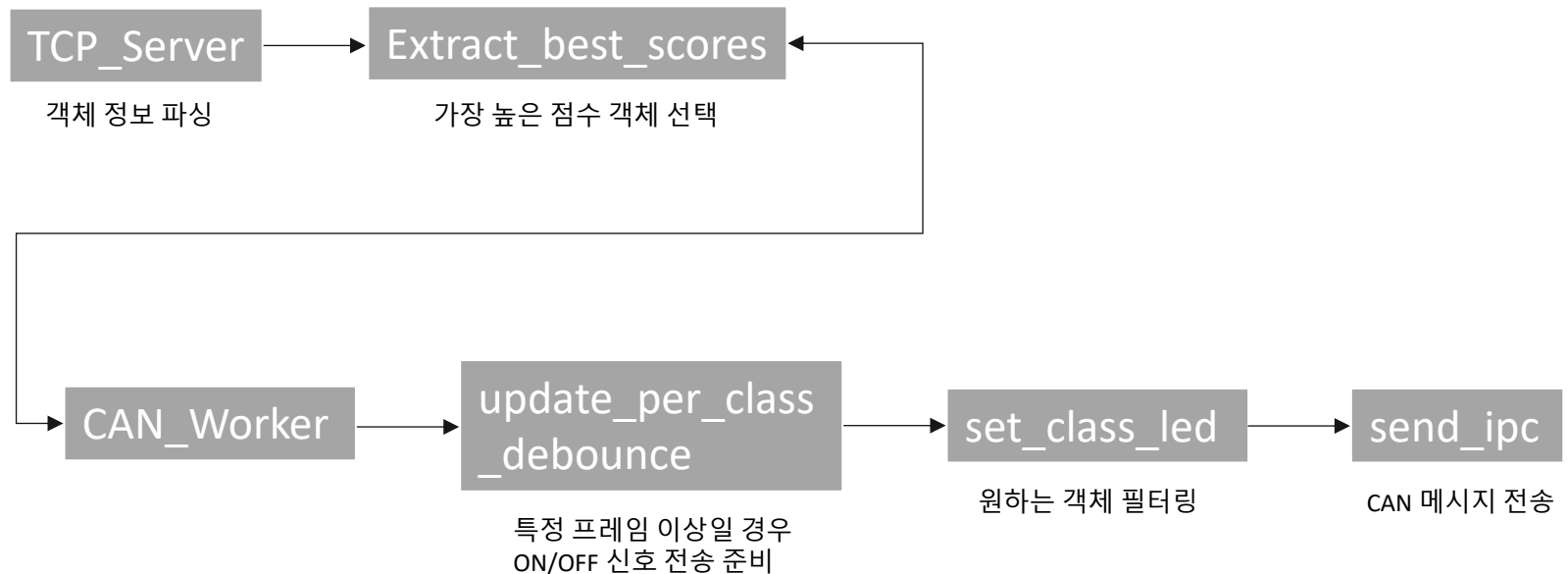
## • 코드 리뷰(D3-G)

1. update\_per\_class\_debounce()
  - 특정 프레임 이상일 경우 모형 차 제어 ON/OFF
2. worker\_tcp\_server()
  - AI-G와 이더넷 통신을 통해 객체 정보를 받아오는 함수
3. worker\_ipc\_sender()
  - 주기적으로 can 메시지를 전송하는 함수
4. start()
  - 스레드를 시작하는 함수

• 코드 경로 - topst\_3day\_course/data\_parsing.py

# TOPST 통합 제어 시스템 코드 리뷰(D3-G)

- 동작 시나리오(D3-G)



# TOPST 통합 제어 시스템 코드 리뷰(VCP-G)

```
void ControlBreadBoardSensors(uint32 mId, uint8 nDataLength, sint8* pucData)
{
    if (nDataLength == 0 || pucData == NULL_PTR)
        return;

    /* 공통: payload[0] 사용 */
    uint8 action = (uint8)pucData[0];

    static uint8 toggle = 0; // 0: left, 1: right
    static sint8 duty = 0;

    if (mId == 0)
    {
        mcu_printf("[LED] unknown class id:0x%x\r\n", (unsigned)mId);
        return;
    }
    LedGpioInitOnce();
    GPIO_Set(CLASS_00, 0);
    GPIO_Set(CLASS_01, 0);
    GPIO_Set(CLASS_02, 0);
    GPIO_Set(CLASS_03, 0);

    if (action == VCP_IO_ACTION_ON){
        switch (mId)
        {
            case CLASS_00:{
                GPIO_Set(CLASS_00, 1);
                uint32 angle = (toggle == 0) ? 40: 150;
                toggle ^= 1;
                Servo_Set_Smooth(angle);
                mcu_printf("[SERVO] angle=%d deg\r\n", angle);
                duty = duty_from_speed(50);
                MotorB_Set(duty, 1);
            }
        }
    }
}
```

## • 코드 리뷰(VCP-G)

### 1. ControlBreadBoardSensors()

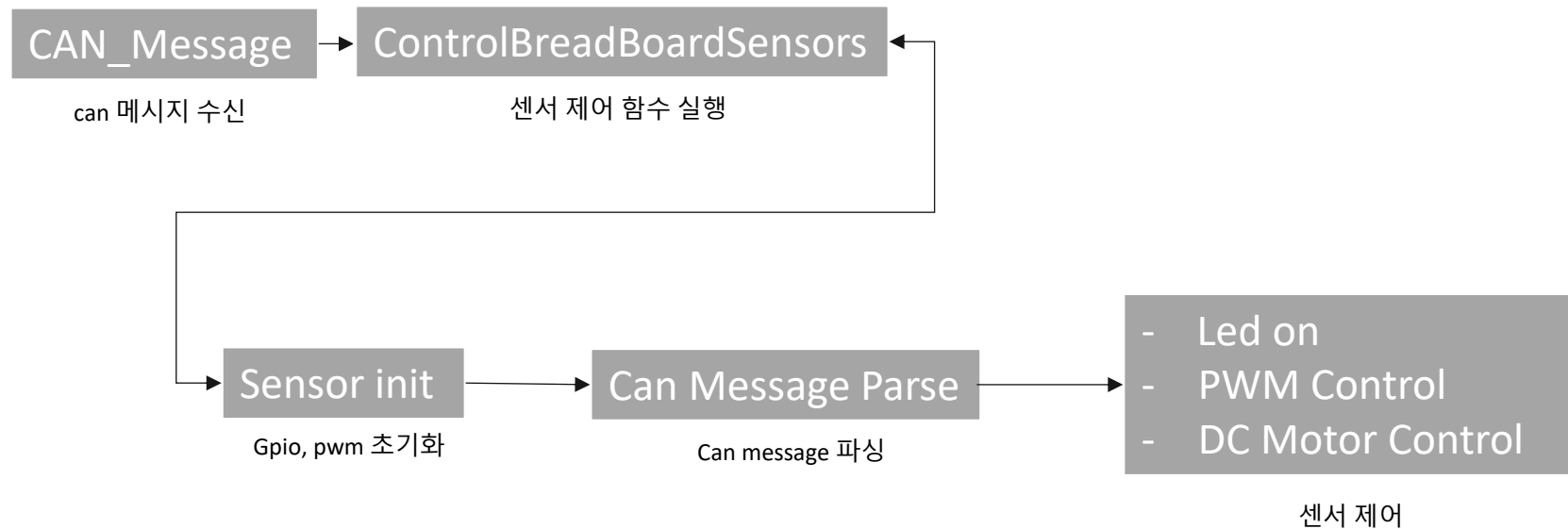
- CAN 메시지 수신 후 파싱
- 파싱된 결과를 바탕으로 센서 제어
- ON 신호를 받을 경우 회피
- ON 신호일때 LED, 서보, DC 모터 제어
- OFF 신호를 받을 경우 복귀
- OFF 신호일때 LED, 서보, DC 모터 제어

- 코드 경로 - FreeRTOS-

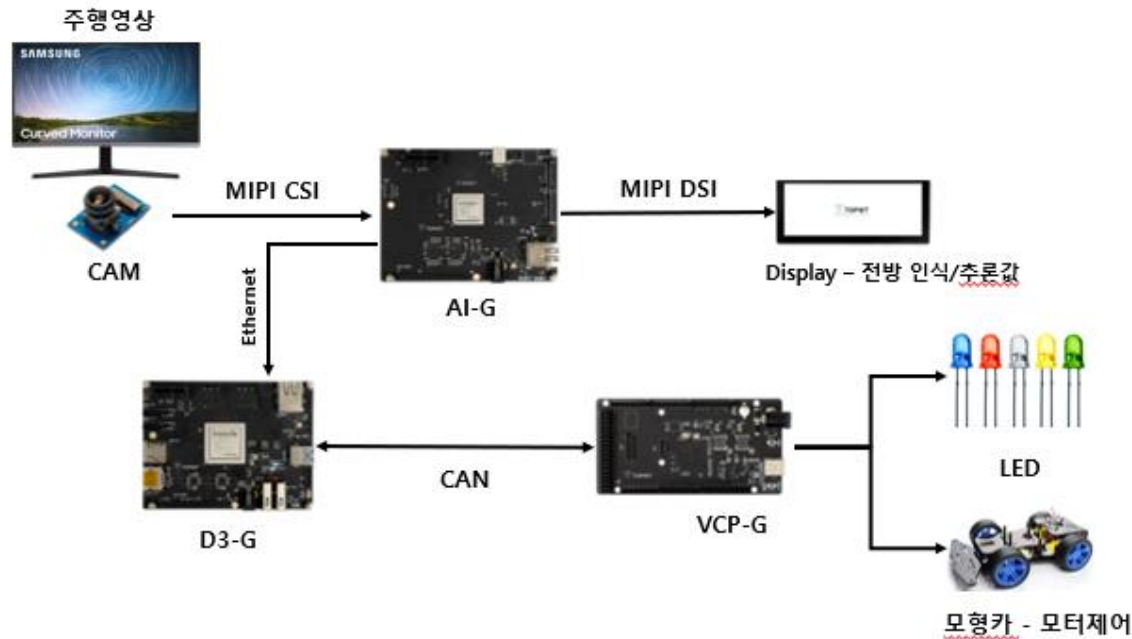
VCP/sources/app.sample/app.can.demo/

# TOPST 통합 제어 시스템 코드 리뷰(VCP-G)

- 동작 시나리오(VCP-G)

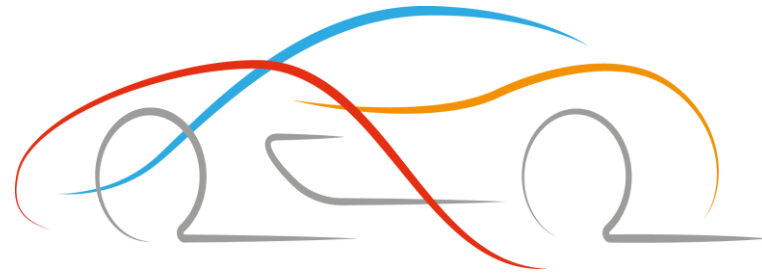


# 사용자 정의 제어 시나리오 구현



## 개인별 추가 기능 구현

- 개인 별 원하는 추가 기능 구현
- EX1) 물체가 감지되면 LCD Display 에 "Object Detected!!" 출력
- EX2) YOLOv8s의 다른 cls에 대한 can message를 구성하고 추가 동작 구현



Thank You